FoundationDB and Its Use for A Graph Database

Hieu Nguyen
Software Engineer, eBay









Outline

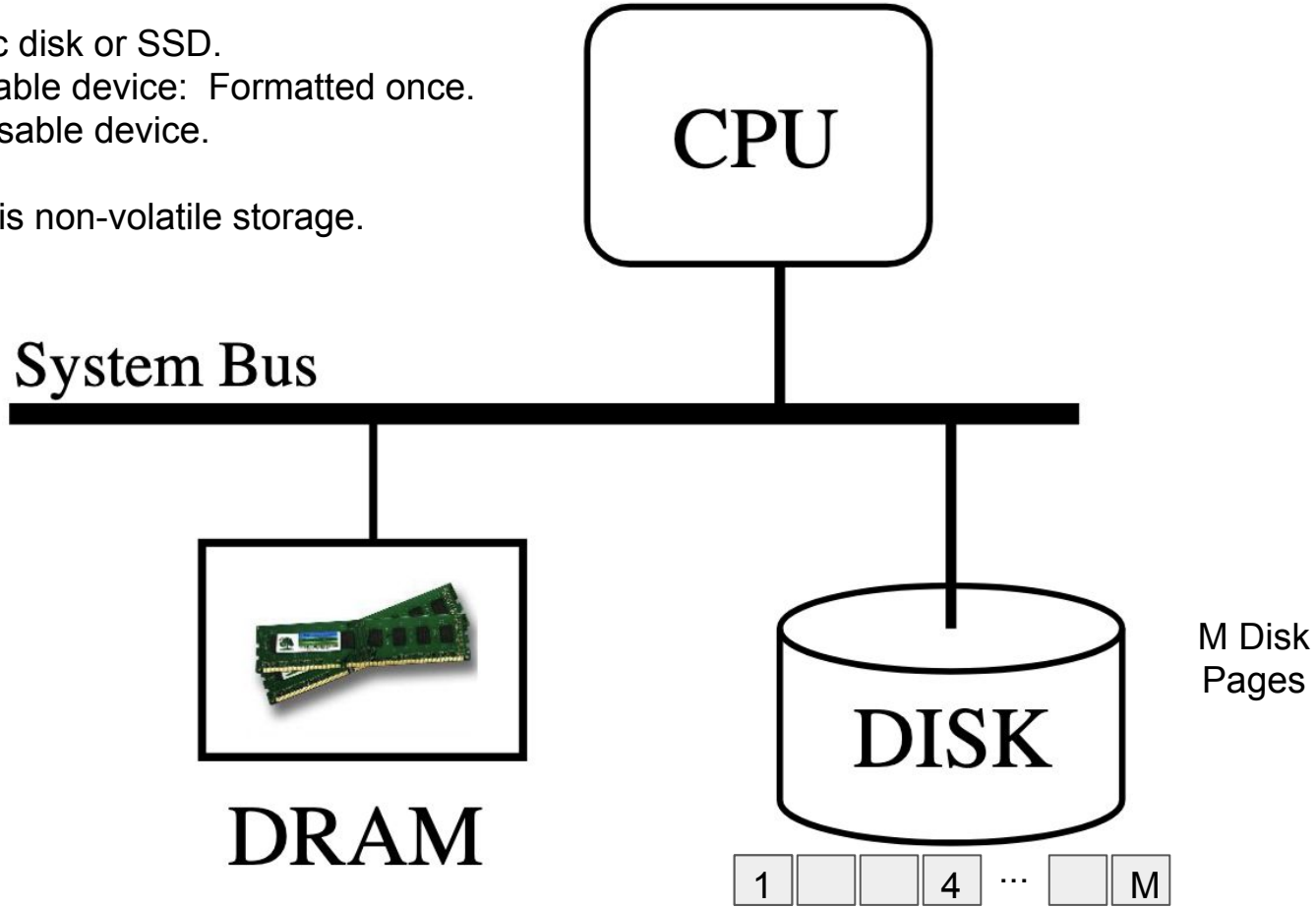
- Part 1. FoundationDB
  - FoundationDB Deployment Architecture
  - FoundationDB Components
  - FoundationDB Transaction
  - Programming with FoundationDB
- Part 2. Graph Database
  - eBay Graph Database Architecture With FDB As the Backend
  - Graph Data Layout and Indices
  - Read and Write with Graph Database
- Part 3. Caching Techniques in Graph Database
  - Full Query Cache Template
  - Per-Hop Query Cache Template
  - Implement Caching Techniques in Graph Database with FoundationDB
  - Multiple Stashes for the Backend

Part 1. FoundationDB

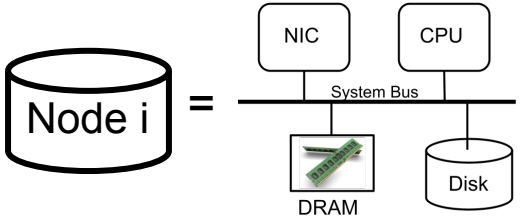
Hardware Organization

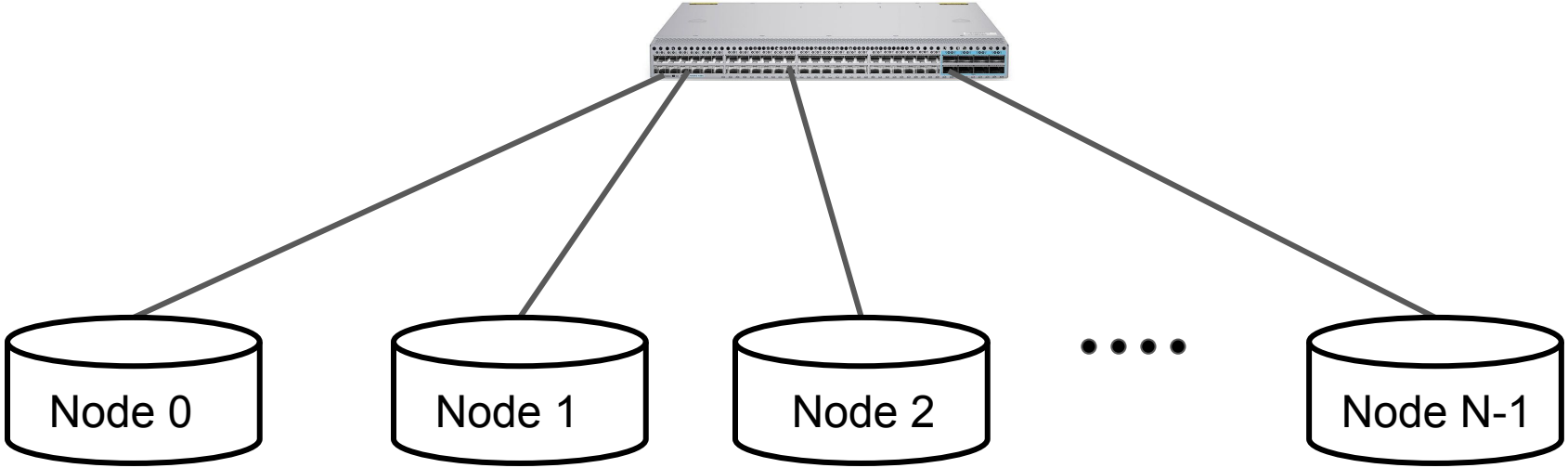
Disk may be a magnetic disk or SSD.
Disk is a block addressable device: Formatted once.
DRAM is a byte-addressable device.

DRAM is volatile. Disk is non-volatile storage.



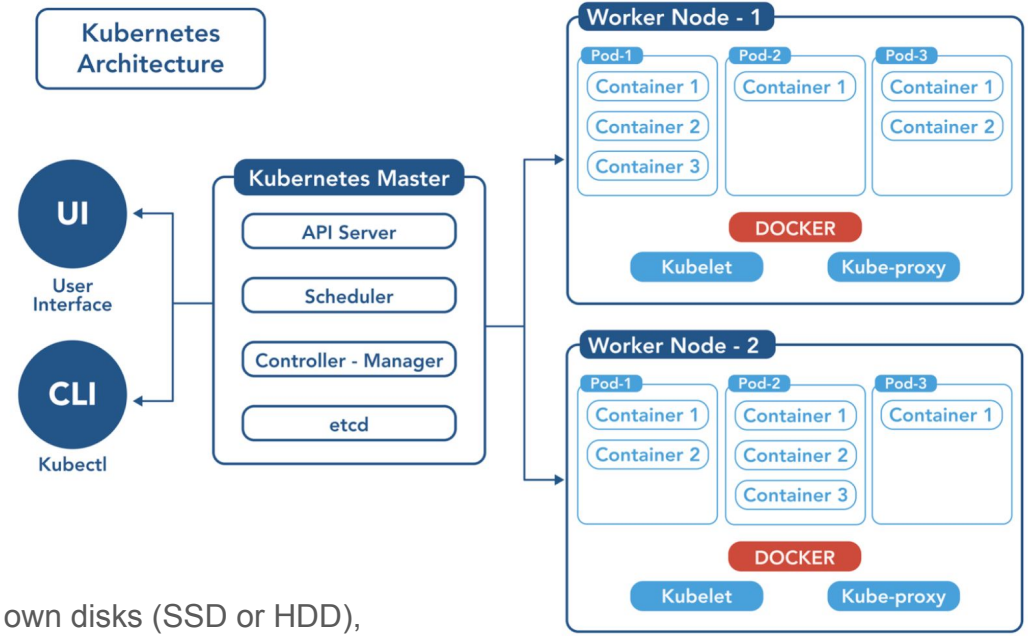
A Shared-Nothing Architecture





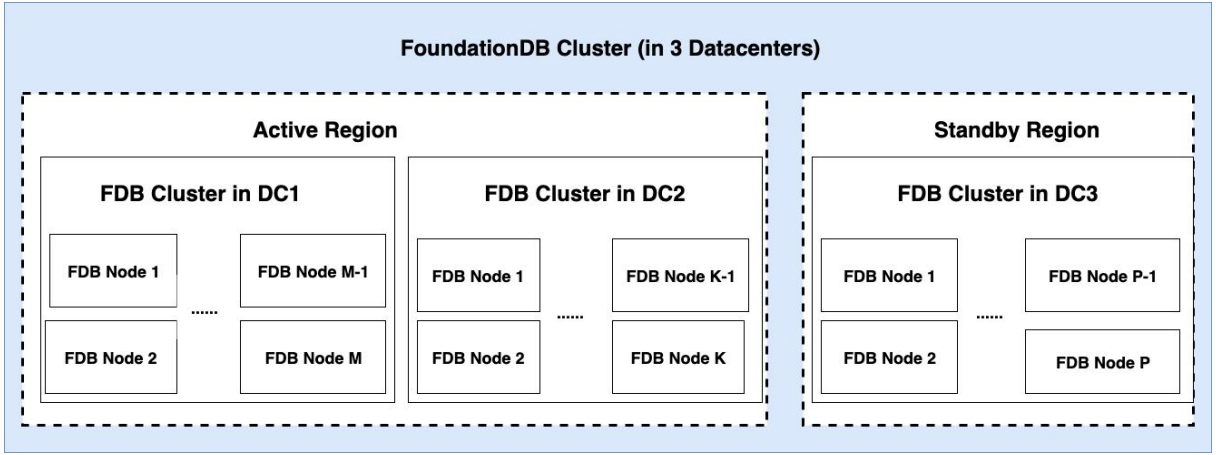
A distributed computing setup where every node has its own independent CPU, memory, and storage, and nodes do not share any physical resources

Kubernetes Architecture (2014)



- Multiple worker nodes
  - Physical (bare metal): Each node has its own disks (SSD or HDD), network, DRAM
  - Virtual machines (Cloud VMs)
- Each node may host multiple pods
- Each pod may have multiple containers

FoundationDB Deployment



- Hundreds of nodes per datacenter
- 3-datacenter, 2-region
  - Cross-DC latency: ~20-30ms
  - Inner-DC latency: 0.2ms
- Tolerate 1 DC Failure
  - DC1 fail - DC2 and DC3 form an active region
- A key-value pair is triple-replicated in each DC
  - 6 copies in total
  - Each DC can tolerate 2 storage node failures

FoundationDB - Distributed Key-Value Store

- Keys are ordered
- Provide **strict serializability**
- MVCC: Multi-version concurrency control
- OCC: Optimistic concurrency control
- Scalable to hundreds of nodes
- Support multiple data-centers
- Layers can be built on top to support other data models (document, relational, graph)
- Acquired by Apple in 2015 and open-sourced in 2018 under the Apache 2.0 license



Databases that provide strict serializability

A few well-known systems offer this guarantee:

- **Google Spanner** – Uses TrueTime API (GPS + atomic clocks) to guarantee global strict serializability across distributed nodes.
- **FoundationDB** – Provides strict serializability by default, even across distributed clusters.
- **Calvin** (deterministic database protocol, used in systems like FaunaDB) – Enforces strict serializability through deterministic transaction ordering.
- **CockroachDB** – Provides strict serializability (linearizable transactions) for committed reads and writes, though some optimizations (like follower reads) relax this.
- **TiDB** – A distributed SQL database that supports strict serializability on transactions using Percolator-like protocol.
- **Single-node systems (PostgreSQL, Oracle, SQL Server)** – In their strongest isolation levels (*serializable*), they provide strict serializability within a single instance (though not distributed across datacenters).

https://www.foundationdb.org/files/fdb-paper.pdf

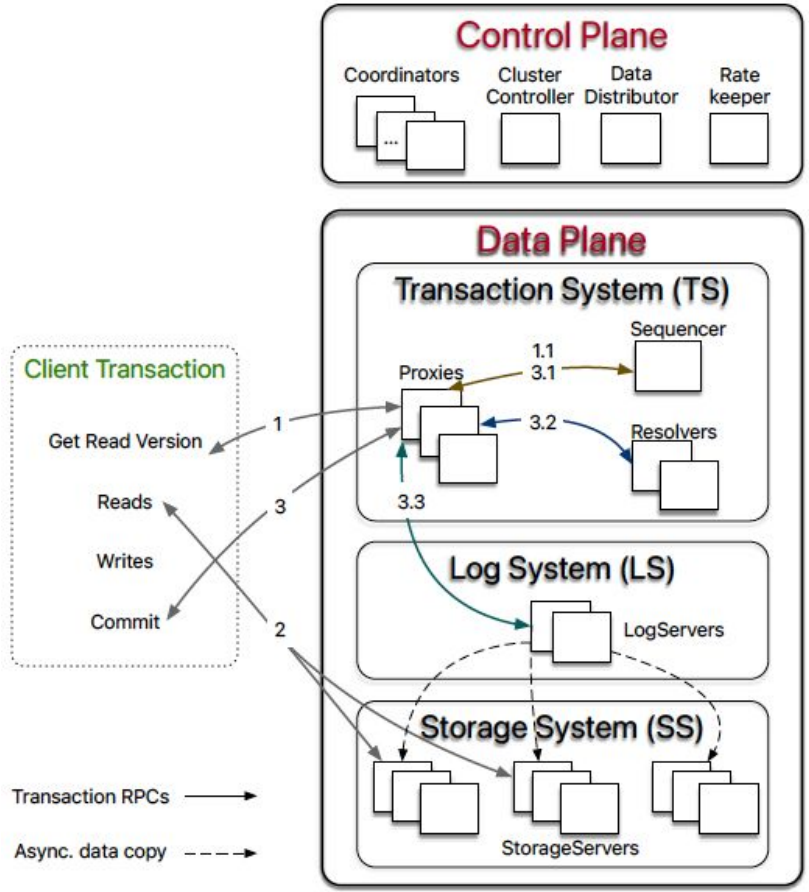
FoundationDB Components - Divide and Conquer

Control Plane

- **Coordinators**:
  - Form a Paxos group
  - Recruit ClusterController, DataDistributor, RateKeeper
- Singletons:
  - **ClusterController**: Monitor all servers
  - **DataDistributor**: Monitor failures and balancing data among StorageServers
  - **RateKeeper**: Overload protection

Data Plane

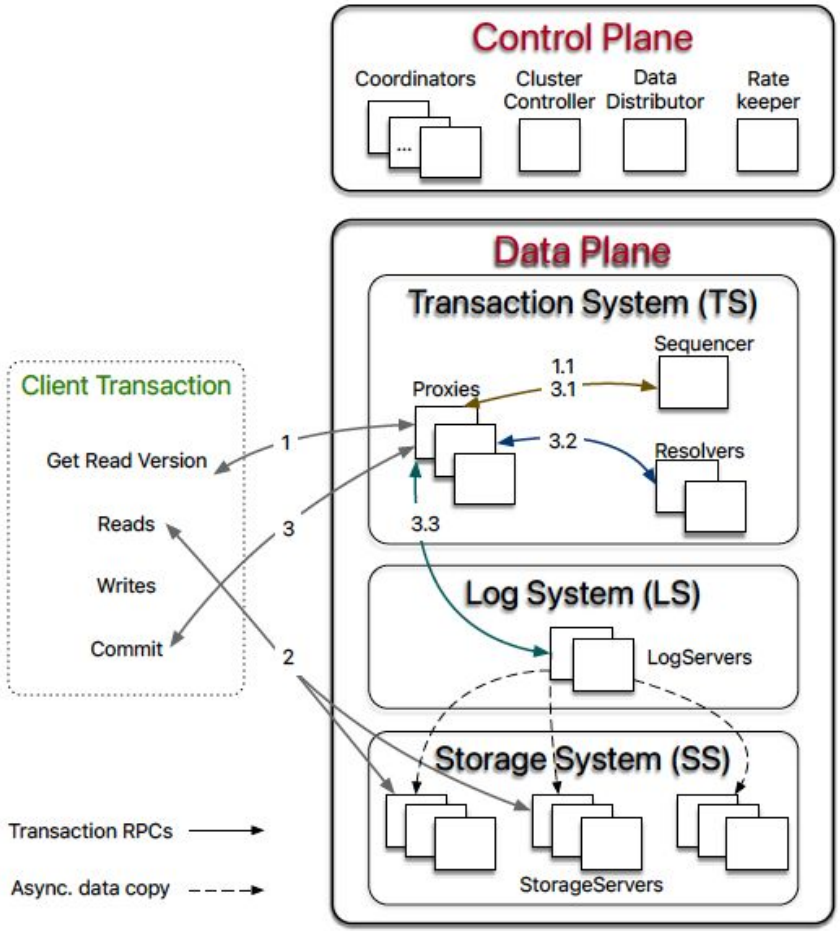
- Transaction System (TS)
  - **Sequencer** (singleton): Assign read and commit versions to transactions
  - **Proxies** (stateless): Offer MVCC read versions to clients and orchestrate transaction commits
  - **Resolvers** (range-based, stateless): Check for conflicts between transactions
- Log System (LS): LogServers
- Storage System (SS): StorageServers



Bootstrapping and Reconfiguration

2.3.4 Bootstrapping FDB has no external dependency on other services. All user data and most of the system metadata (keys that start with `0xFF` prefix) are stored in `StorageServers`. The metadata about `StorageServers` is persisted in `LogServers`, and the configuration of LS (i.e., information about `LogServers`) is stored in all `Coordinators`. Using `Coordinators` as a disk Paxos group, servers attempt to become the `ClusterController` if one does not exist. The newly elected `ClusterController` recruits a new `Sequencer`, which reads the configuration of old LS stored in `Coordinators` and spawns a new TS and LS. From the old LS, `Proxies` recover system metadata, including information about all `StorageServers`. The `Sequencer` waits until the new TS finishes recovery (see Section 2.4.4), and then writes the new LS configuration to all `Coordinators`. At this time, the new transaction system becomes ready to accept client transactions.

2.3.5 Reconfiguration Whenever there is a failure in the TS or LS, or a database configuration change, a reconfiguration process brings the transaction management system to a new configuration, i.e., a clean state. Specifically, the `Sequencer` process monitors the health of `Proxies`, `Resolvers`, and `LogServers`. If any one of the monitored processes fails or the database configuration changes, the `Sequencer` process terminates. The `ClusterController` will detect the `Sequencer` failure event, then recruit a new `Sequencer`, which follows the above bootstrapping process to spawn the new TS and LS instance. In this way, transaction processing is divided into epochs, where each epoch represents a generation of the transaction management system with its unique `Sequencer` process.



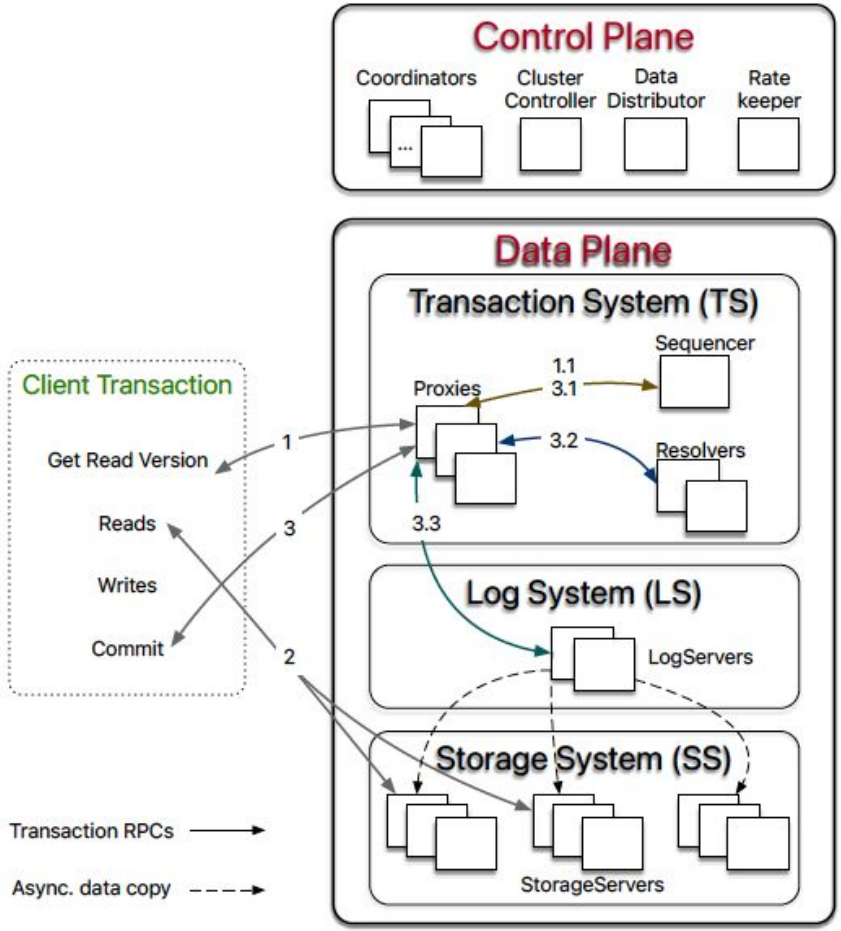
FDB Transaction (OCC)

GetReadVersion

- (1) The client sends a request to a ProxyServer asking for a read version
- (1.1) The proxy gets the read version from Sequencer
  - It must be greater than the last committed version

Reads

- (2) The client issues reads directly to StorageServers
  - Cache the mapping between key ranges and storage servers



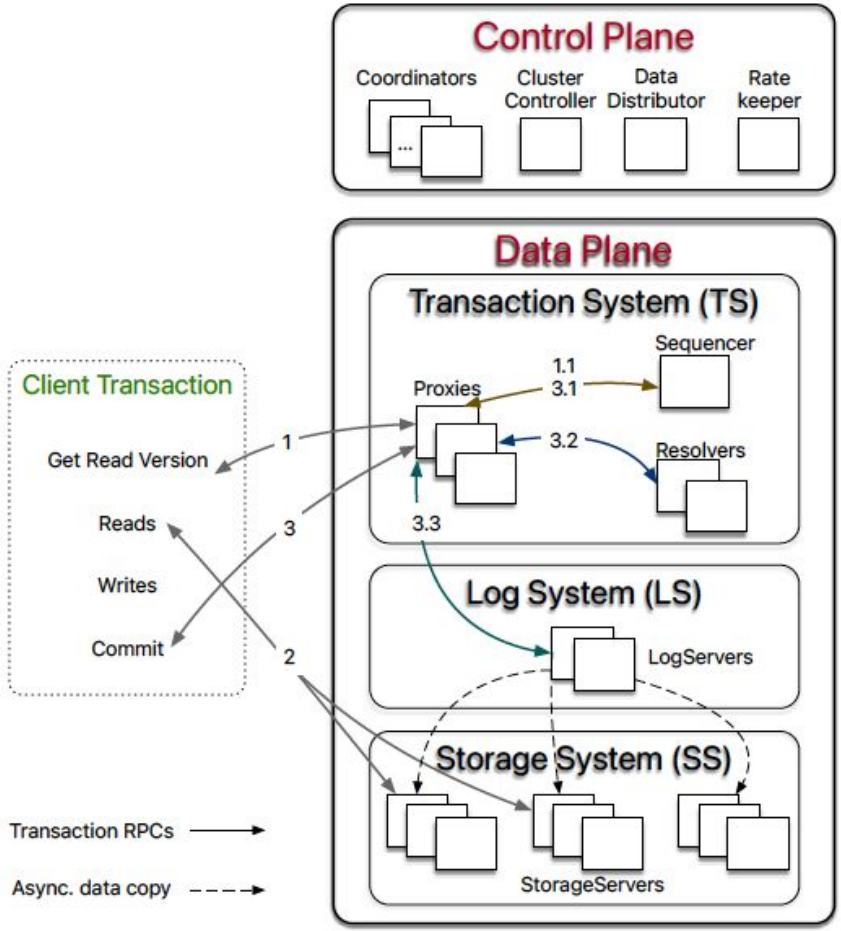
FDB Transaction (cont.)

Writes

- Buffer locally at the client

Commit

- (3) The client sends read and write sets to a ProxyServer
- (3.1) The Proxy asks the Sequencer for a commit version
- (3.2) The proxy asks the resolvers to check for read-write conflicts
  - Transaction can abort if a conflict is detected
- (3.3) The Proxy sends committed mutations to log servers for persistence.
  - A transaction is considered committed if the required log servers reply to the Proxy
- (3.4) The Proxy reports the committed version to the Sequencer before returning success to the client.



FDB Resolver - Conflict Checking

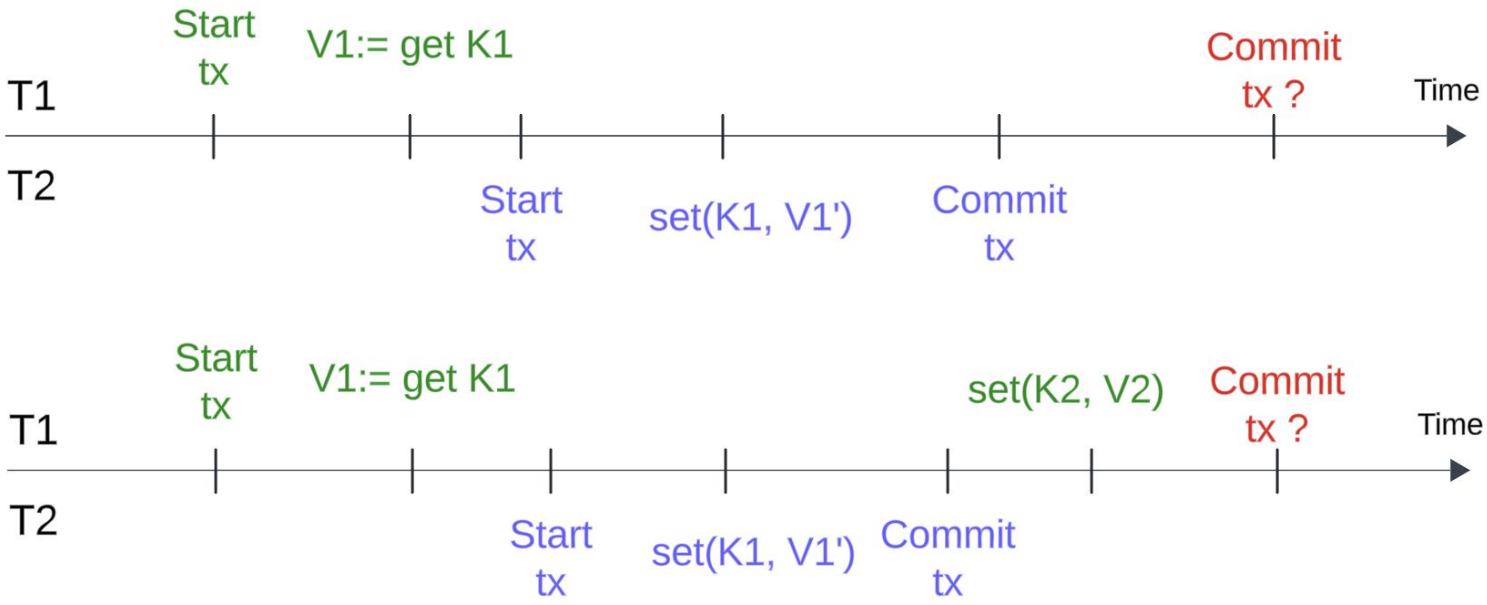
Algorithm 1 illustrates the lock-free conflict detection algorithm on `Resolvers`. Specifically, each `Resolver` maintains a history *lastCommit* of recently modified key ranges by committed transactions, and their corresponding commit versions. The commit request for T_x comprises two sets: a set of modified key ranges R_w, and a set of read key ranges R_r, where a single key is converted to a single key range. The read set is checked against the modified key ranges of concurrent committed transactions (line 1—5), which prevents phantom reads. If there are no read-write conflicts, `Resolvers` admit the transaction for commit and update the list of modified key ranges with the write set (line 6—7). For snapshot reads, they are not included in the set R_r. In practice, *lastCommit* is represented as a version-augmented probabilistic SkipList [56].

Algorithm 1: Check conflicts for transaction T_x.

Require: *lastCommit*: a map of key range → last commit version

```
1 for each range ∈ R_r do
2     ranges = lastCommit.intersect(range)
3     for each r ∈ ranges do
4         if lastCommit[r] > T_x.readVersion then
5             return abort;
  // commit path
6 for each range ∈ R_w do
7     lastCommit[range] = T_x.commitVersion;
8 return commit;
```

T1 Commits or Aborts?



FDB Logging Protocol

2.4.3 Logging Protocol After a Proxy decides to commit a transaction, the log message is broadcast to all LogServers. As illustrated in Figure 2, the Proxy first consults its in-memory shard map to determine the StorageServers responsible for the modified key range. Then the Proxy attaches StorageServer tags 1, 4, and 6 to the mutation, where each tag has a preferred LogServer for storage. In this example, tags 1 and 6 have the same preferred LogServer. Note the mutation is only sent to the preferred LogServers (1 and 4) and an additional LogServer 3 to meet the replication requirements. All other LogServers receive an empty message body. The log message header includes both LSN and the previous LSN obtained from the Sequencer, as well as the known committed version (KCV) of this Proxy. LogServers reply to the Proxy once the log data is made durable, and the Proxy updates its KCV to the LSN if all replica LogServers have replied and this LSN is larger than the current KCV.

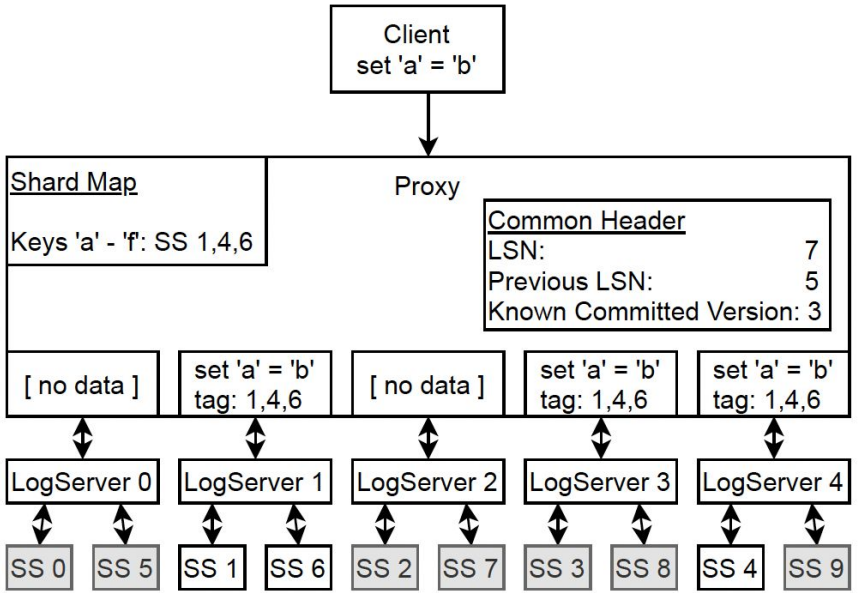


Figure 2: Proxy writes a client mutation to LogServers after sequencing and resolution. Later, the mutation will be asynchronously replicated to StorageServers.

FDB Storage Server

- Storage server maintains versions of data in the last 5 seconds in-memory
- The latest applied value of each key is stored persistently on the configured storage engine (e.g. SQLite)

How FDB Storage Server Serves a Read Request?

LastAppliedVersion: 10
OldestVersion: 1



Get(X, 4): The largest version <= 4 is chosen, i.e. version 3 of key X

Get(Y, 1): Key not found

Get (X, 6): Version 6 is chosen

Get(Y, 8): Version 7 is chosen

Get(Y, 11): Error("Future version")

If OldestVersion = 2:

Get(Y, 1): Error("Version too old"), as the storage server supports reads only in [2, 10]

Storage Server Serving a Read Request

What if a StorageServer is lagging behind on applying the redo logs and a client requests a version of a key pair it does not have?

1. Wait for a threshold for when known-committed-version is greater than or equal to the read version
2. If the request times out, the client asks another StorageServer that stores the key
3. Return error "request for a future version" (FDB error code 1009)

What if there are no more transaction logs to replay?

- Without new transactions issued from the client, Commit proxies still generate empty transactions to advance the known-committed-version
- Known-committed-version and LSN of each transaction are sent to all LogServers (limit scalability on writes)

Recovery from Failure

- The recovery process starts by detecting a failure, recruits a new transaction system, and ends when old LogServers are no longer needed.
- **KCV** (Known-committed version): The maximum LSN that this Proxy has committed
- **DV** (Durable version): The maximum persisted LSN
- **PEV** (Epoch's end version): maximum of all KCV
- **RV** (Recovery's version): Minimum of all DV
- Logs in the range of [PEV +1,RV] are copied from previous epoch's LogServers to the current ones, for healing the replication degree in case of LogServer failures.

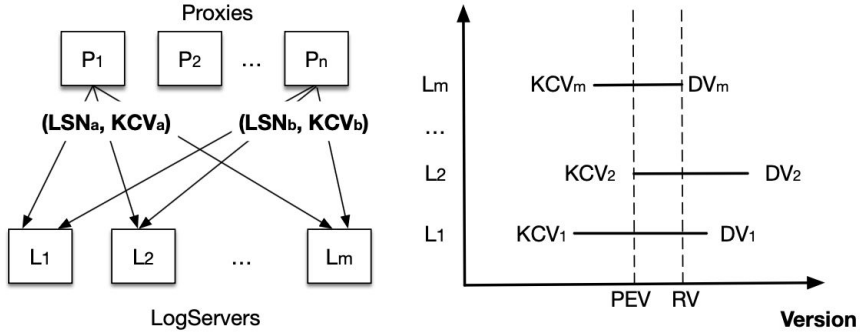


Figure 4: An illustration of RV and PEV. On the left, a `Proxy` sends redo logs to `LogServers` with a KCV and the LSN, and `LogServers` keep the maximum KCV received. On the right, recovery uses the maximum of KCVs and the minimum of DVs on a set of `LogServers` as PEV and RV, respectively.

When `Sequencer` accepts new transactions, the first is a special recovery transaction that informs `StorageServers` the RV so that they can roll back any data larger than RV. The current FDB storage engine consists of an unversioned SQLite [41] B-tree and in-memory multi-versioned redo log data. Only mutations leaving the MVCC window (i.e., committed data) are written to SQLite. The rollback is simply discarding in-memory multi-versioned data in `StorageServers`. Then `StorageServers` pull any data larger than version PEV from new `LogServers`.

Simulation Testing

Deterministic simulator. FDB was built from the ground up to make this testing approach possible. All database code is deterministic; accordingly multithreaded concurrency is avoided (instead, one database node is deployed per core). Figure 6 illustrates the simulator process of FDB, where all sources of nondeterminism and communication are abstracted, including network, disk, time, and pseudo random number generator. FDB is written in Flow [4], a novel syntactic extension to C++ adding async/await-like concurrency primitives. Flow provides the Actor programming model [13] that abstracts various actions of the FDB server process into a number of actors that are scheduled by the Flow runtime library. The simulator process is able to spawn multiple FDB servers that communicate with each other through a simulated network in a single discrete-event simulation. The production implementation is a simple shim to the relevant system calls.

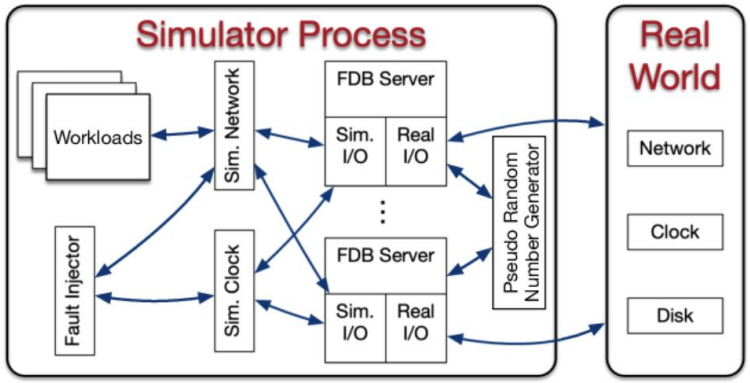


Figure 6: The FDB deterministic simulator.

Benefit: once a bug is detected, it can always be reproducible

Fault Injection

Fault injection. The FDB simulator injects machine, rack, and data-center level fail-stop failures and reboots, a variety of network faults, partitions, and latency problems, disk behavior (e.g. the corruption of unsynchronized writes when machines reboot), and randomizes event times. This variety of fault injection both tests the database's resilience to specific faults and increases the diversity of states in simulation. Fault injection distributions are carefully tuned to avoid driving the system into a small state-space caused by an excessive fault rate.

```
wait(c);  // Wait for persistentQueue->commit() to finish

if (g_network->isSimulated() &&
    !g_simulator.speedUpSimulation &&
    BUGGIFY_WITH_PROB(0.0001)) {
    wait(delay(6.0));
}
```

FoundationDB - Quiz

What does Commit do in a read-only transaction (no write)?

The client skips the commit protocol if there is no write.

Is it possible multiple resolvers resolve conflicts for a transaction?

Yes.

What are the consequences if one resolver detects a conflict while the others do not?

The transaction is aborted and other resolvers may potentially false-positive detect conflicts on future transactions. This is a fase-positive conflict and does not violate correctness.

FDB Programming Example

Emp

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1
555	Alice	45	80000	1
666	Bob	50	90000	2

Dept

dno	dname	floor
1	shoe	1
2	toy	1
3	men	2
4	women	2

FDB Programming Core API

GET Read single key

```
CompletableFuture<byte[]> get(byte[] key)
```

SET Write key-value pair

```
void set(byte[] key, byte[] value)
```

GET RANGE Range query iterators

```
AsyncIterable<KeyValue> getRange(byte[] start,
byte[] end)
AsyncIterable<KeyValue> getRange(Range range)
```

CLEAR Delete key or range

```
void clear(byte[] key)
void clear(byte[] start, byte[] end)
void clear(Range range)
```

FDB Programming Example (cont.)

```
public static Emp[] emps = new Emp[] {  1 usage
        new Emp() {{ssNum = "123"; name = "Mary"; age = 22; salary = 55000; dNo = 3;}},
        new Emp() {{ssNum = "323"; name = "Joe"; age = 33; salary = 34000; dNo = 1;}},
        new Emp() {{ssNum = "324"; name = "John"; age = 40; salary = 55000; dNo = 2;}},
        new Emp() {{ssNum = "333"; name = "Mike"; age = 25; salary = 60000; dNo = 1;}},
        new Emp() {{ssNum = "555"; name = "Alice"; age = 45; salary = 80000; dNo = 1;}},
        new Emp() {{ssNum = "666"; name = "Bob"; age = 50; salary = 90000; dNo = 2;}},
};

public static Dept[] depts = new Dept[] {  1 usage
        new Dept() {{dNo = 1; dName = "shoe"; floor = 1;}},
        new Dept() {{dNo = 2; dName = "toy"; floor = 1;}},
        new Dept() {{dNo = 3; dName = "men"; floor = 2;}},
        new Dept() {{dNo = 4; dName = "women"; floor = 2;}},
};
```

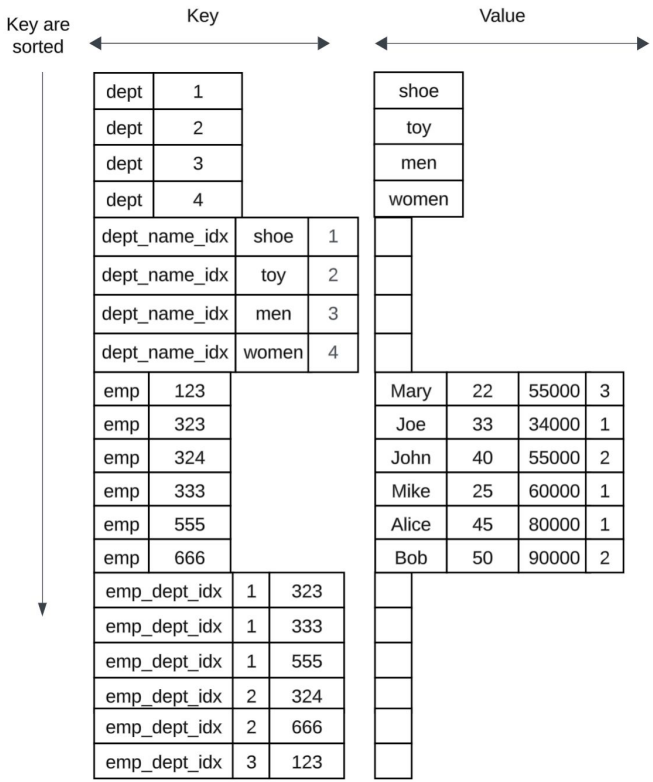
FDB Programming Example - Load Data

```
// Load emps and build index
db.run(tr -> {
    for (Emp emp: emps) {
        Tuple empKey = Tuple.from( ...items: "emp", emp.ssNum);
        tr.set(empKey.pack(), Tuple.from(emp.name, emp.age, emp.salary, emp.dNo).pack());

        // Build index for emp by dNo
        Tuple empIdxKey = Tuple.from( ...items: "emp_dept_idx", emp.dNo, emp.ssNum);
        tr.set(empIdxKey.pack(), Tuple.from().pack());
    }
    return null;
});

// Load depts and build index
db.run(tr -> {
    for (Dept dept: depts) {
        Tuple deptKey = Tuple.from( ...items: "dept", dept.dNo);
        tr.set(deptKey.pack(), Tuple.from(dept.dName, dept.floor).pack());

        // Build index for dept by dName
        Tuple deptIdxKey = Tuple.from( ...items: "dept_name_idx", dept.dName, dept.dNo);
        tr.set(deptIdxKey.pack(), Tuple.from().pack());
    }
    return null;
});
```

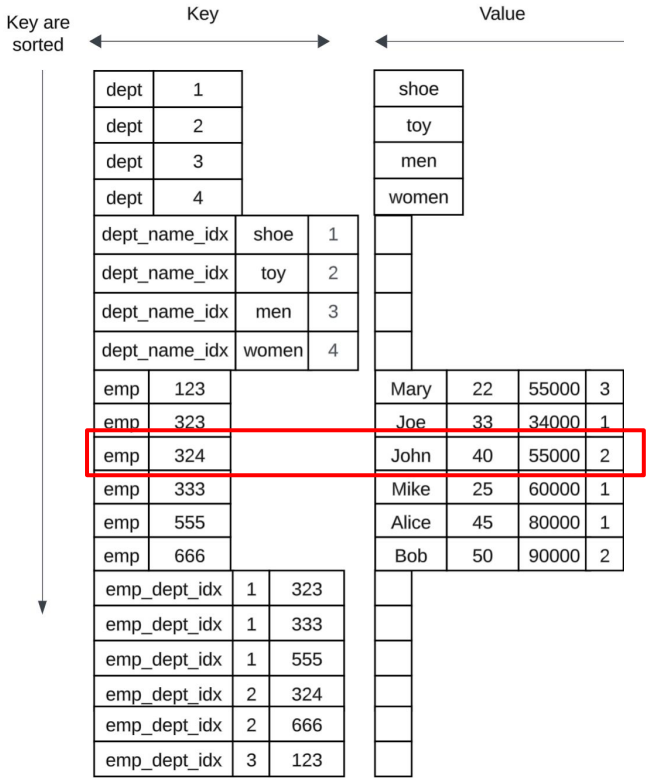


FDB - Queries

Get an employee by SSN

getEmp(db, 324)

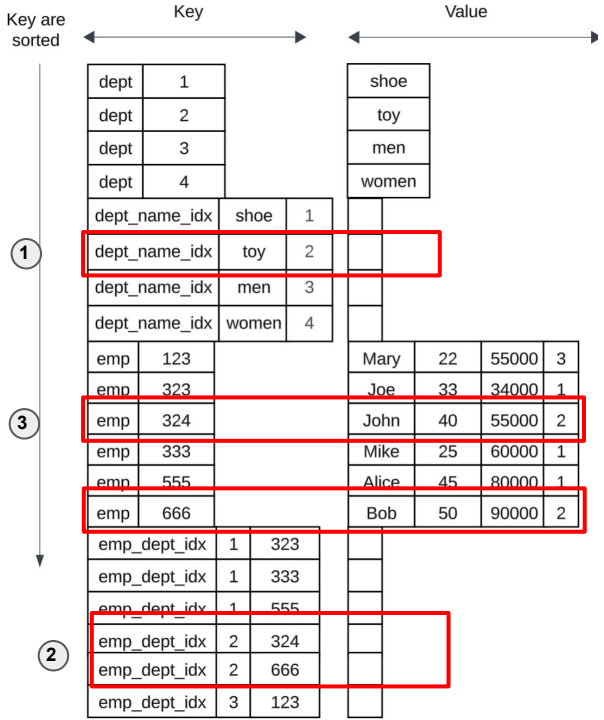
```
public static void getEmp(Database db, String ssNum) {  no usages
    db.run(tr -> {
        Tuple empKey = Tuple.from( ...items: "emp", ssNum);
        byte[] empVal = tr.get(empKey.pack()).join();
        Tuple empValTuple = Tuple.fromBytes(empVal);
        String name = empValTuple.getString( index: 0);
        int age = empValTuple.getBigInteger( index: 1).intValue();
        long salary = empValTuple.getLong( index: 2);
        log.info("Emp: name={}, age={}, salary={}", name, age, salary);
        return null;
    });
}
```



FDB - Queries (cont.)

Get all employees in a department by department name

getEmpsByDeptName(db, "toy")




```java
public static void getEmpsByDept(Transaction tr, int dNo) {  1 usage
    Tuple empIdxKey = Tuple.from( ...items: "emp_dept_idx", dNo);
    Range range = Range.startsWith(empIdxKey.pack());
    List<KeyValue> empIdxVals = tr.getRange(range).asList().join();
    for (KeyValue kv: empIdxVals) {
        // Extract the emp ssNum
        Tuple empIdxKeyTuple = Tuple.fromBytes(kv.getKey());
        String ssNum = empIdxKeyTuple.getString( index: 2);
        getEmp(tr, ssNum);
    }
}

public static void getEmpsByDeptName(Database db, String dName) {  1 usage
    db.run(tr -> {
        Tuple deptIdxKey = Tuple.from( ...items: "dept_name_idx", dName);
        Range range = Range.startsWith(deptIdxKey.pack());
        List<KeyValue> deptIdxVals = tr.getRange(range).asList().join();
        for (KeyValue kv: deptIdxVals) {
            // Extract the dept dNo
            Tuple deptIdxKeyTuple = Tuple.fromBytes(kv.getKey());
            int dNo = deptIdxKeyTuple.getBigInteger( index: 2).intValue();
            getEmpsByDept(tr, dNo);
        }
        return null;
    });
}
```

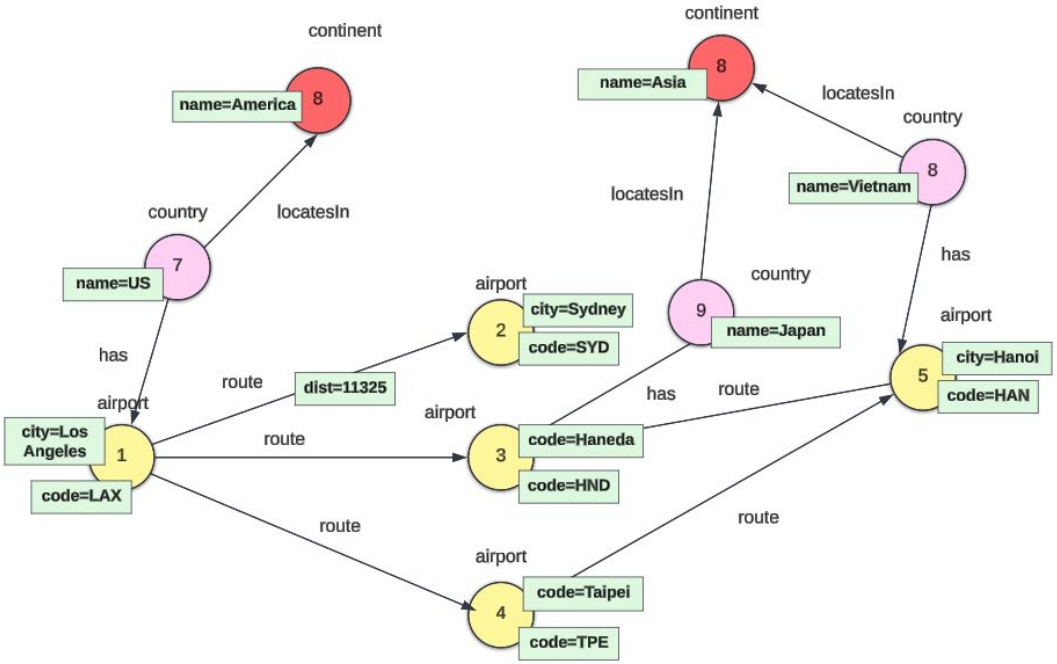
Data Models Built on FoundationDB

- Relational:
  - FDB Record Layer: https://arxiv.org/abs/1901.04452
- Document:
  - https://foundationdb.github.io/fdb-document-layer/
- Replacing Zookeeper:
  - FDBKeeper: Enabling Scalable Coordination Services for Metadata Management using Distributed Key-Value Databases, https://www.vldb.org/pvldb/vol18/p5004-cai.pdf
- **Graph:**
  - **JanusGraph that uses FDB as its storage backend**

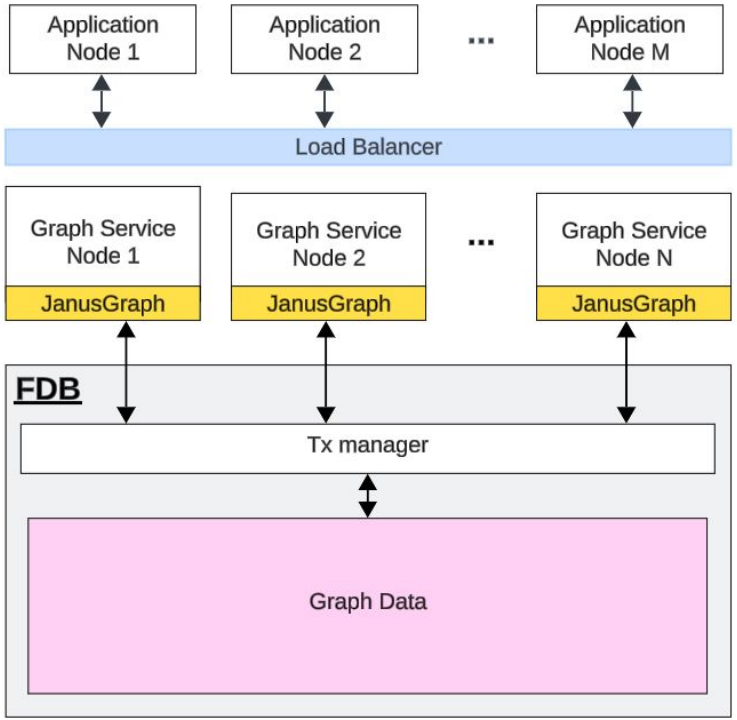
Part 2. Graph Database

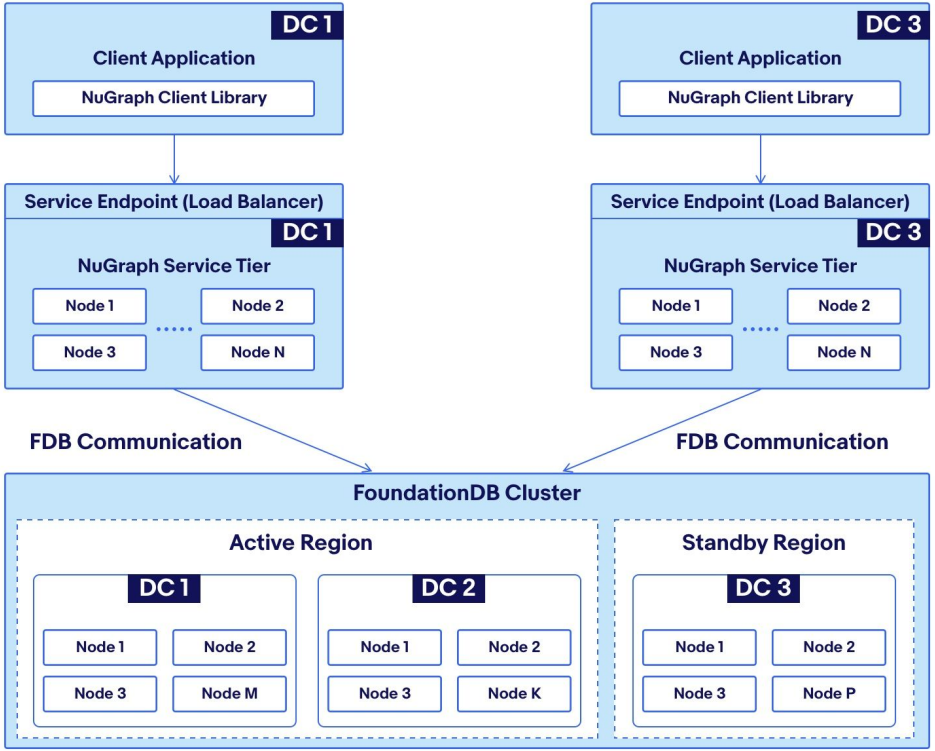
A Graph Example

- Graph elements include vertices (nodes) and edges
- Each vertex or edge has one label and zero or more properties
- An edge connects two vertices



Graph Service Architecture





JanusGraph





- Open source
- Graph database engine
  - Storage backend: FoundationDB, Cassandra, HBase, BerkeleyDB, etc.
  - TinkerPop: A graph computing framework with Gremlin query language
- Horizontally scalable
- Designed to manage billions of vertices and edges

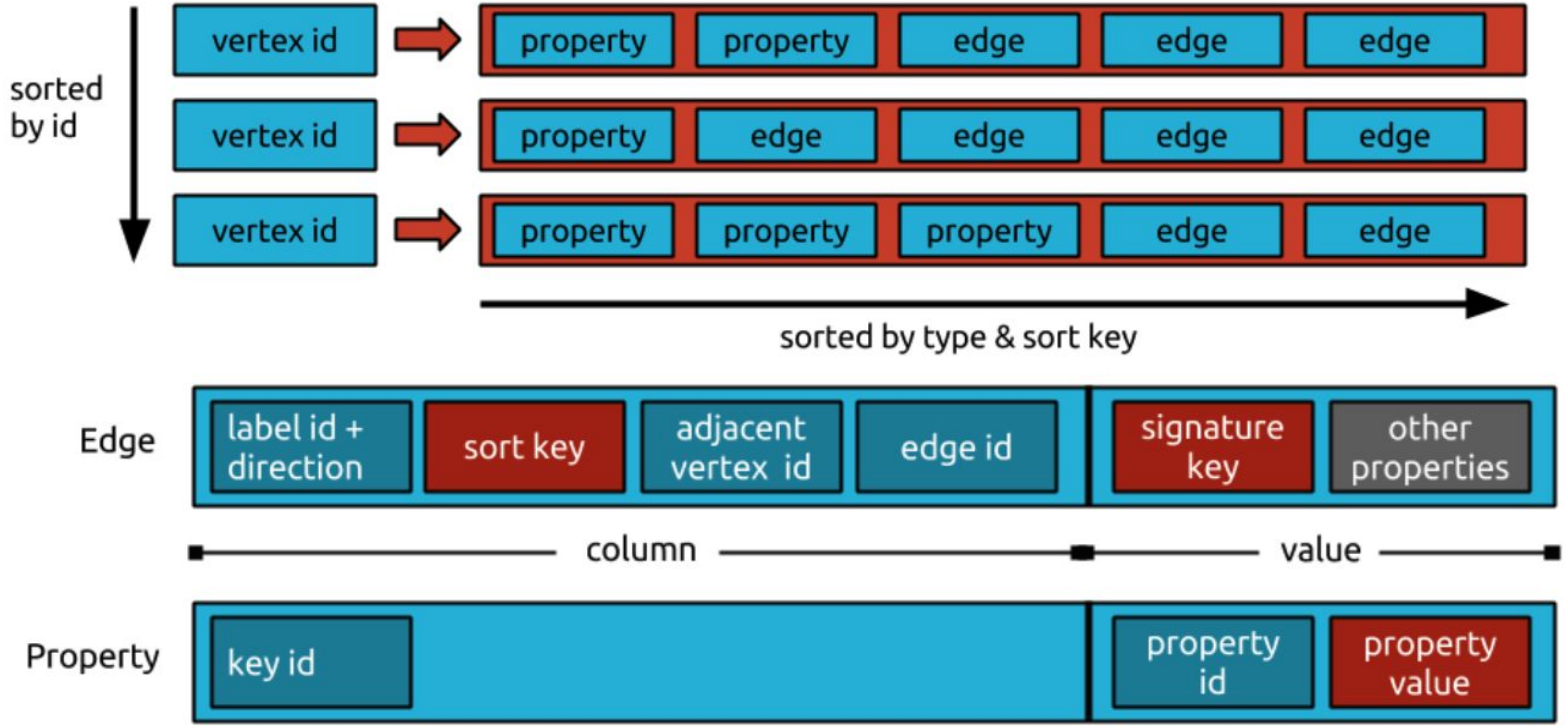
Gremlin query examples

- Find an airport with code "LAX" and retrieve all its properties
  - g.V().has("airport", "code", "LAX").valueMap()
- Find all airports that "LAX" can directly connect to
  - g.V().has("airport", "code", "LAX").outE("route").inV().valueMap()
- Find all airports that connect directly to "LAX"
  - g.V().has("airport", "code", "LAX").inE("route").outV()
- Find all possible 2-hop flights from Hanoi to Los Angeles

```
g.V().has('airport', 'city', 'Hanoi')
     .out('route').out('route').has('city', 'Los Angeles')
     .path()
          .by('code')
          .by(valueMap('city','code'))
          .by('code')

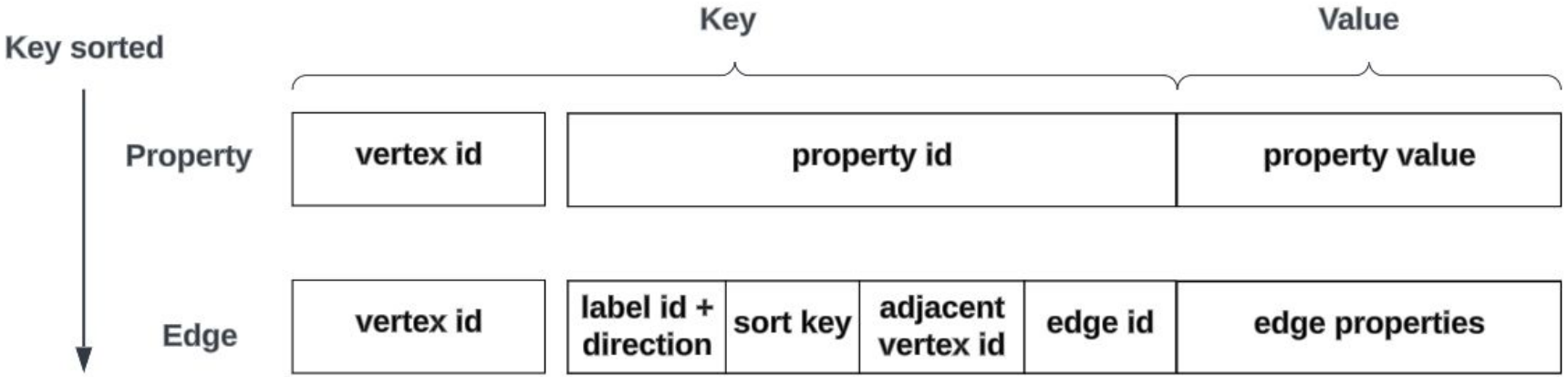
==>path[HAN, {code=[PVG], city=[Shanghai]}, LAX]
==>path[HAN, {code=[CDG], city=[Paris]}, LAX]
==>path[HAN, {code=[SYD], city=[Sydney]}, LAX]
...
```

JanusGraph Data Layout - Key-Column-Value



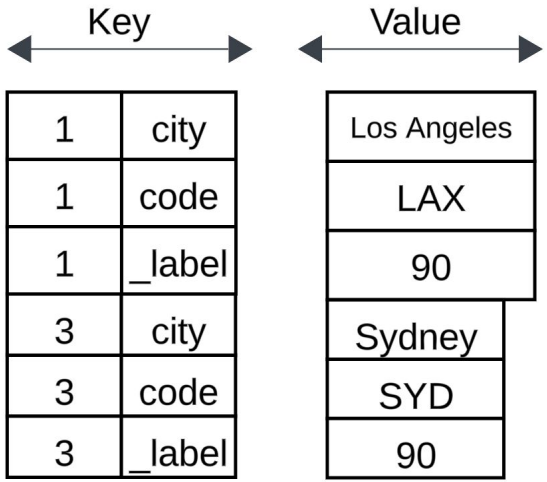
https://docs.janusgraph.org/advanced-topics/data-model/

JanusGraph Data Layout - Key-Value



Graph Operation: Add a Vertex

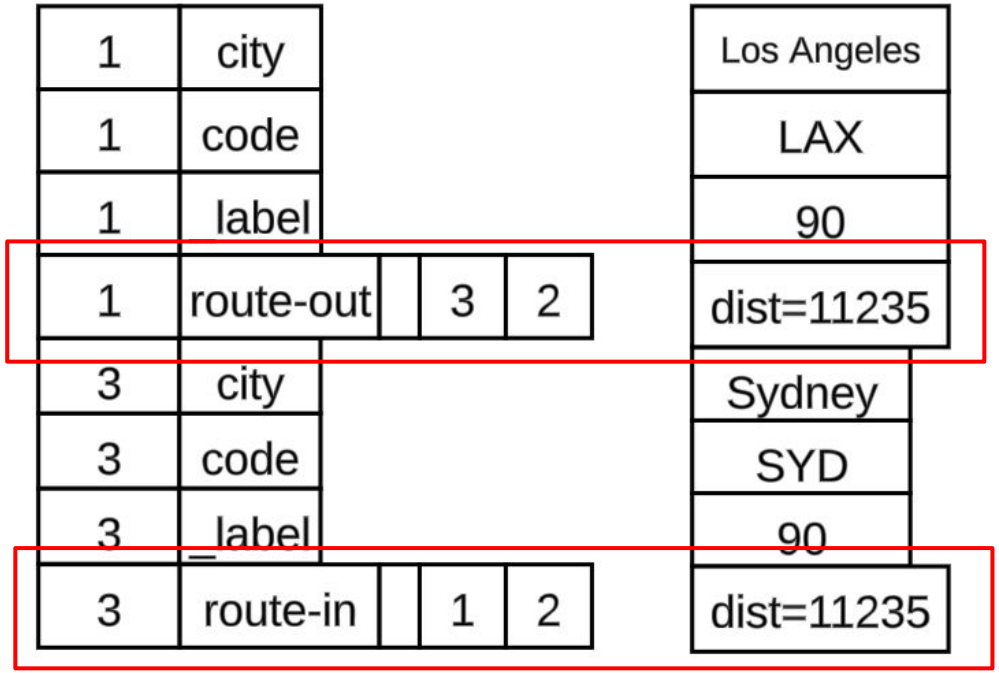
```
g.addV("airport")
        .property("city", "Los Angeles")
        .property("code", "LAX")
g.addV("airport")
        .property("city", "Sydney")
        .property("code", "SYD")
```



Graph Operation: Add an Edge

```
g.addE("route")
    .property("dist", 11235)
    .from(V(1)).to(V(3))
```

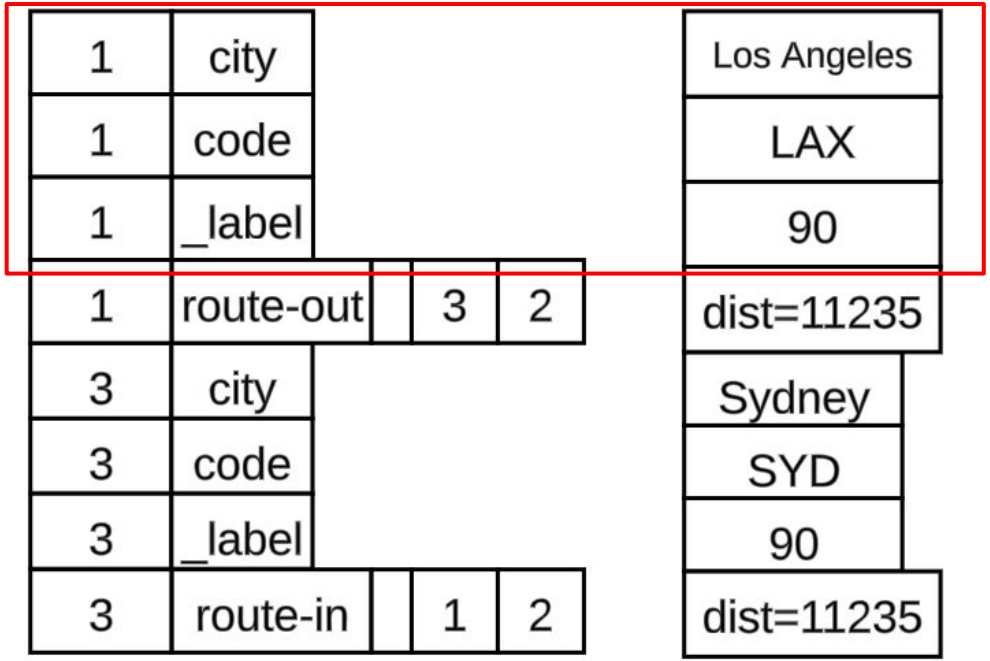
- "2" is the edge id in this example
- Add two keys that represent this edge



Graph Operation: Get all Vertex Properties

g.V(1).valueMap()

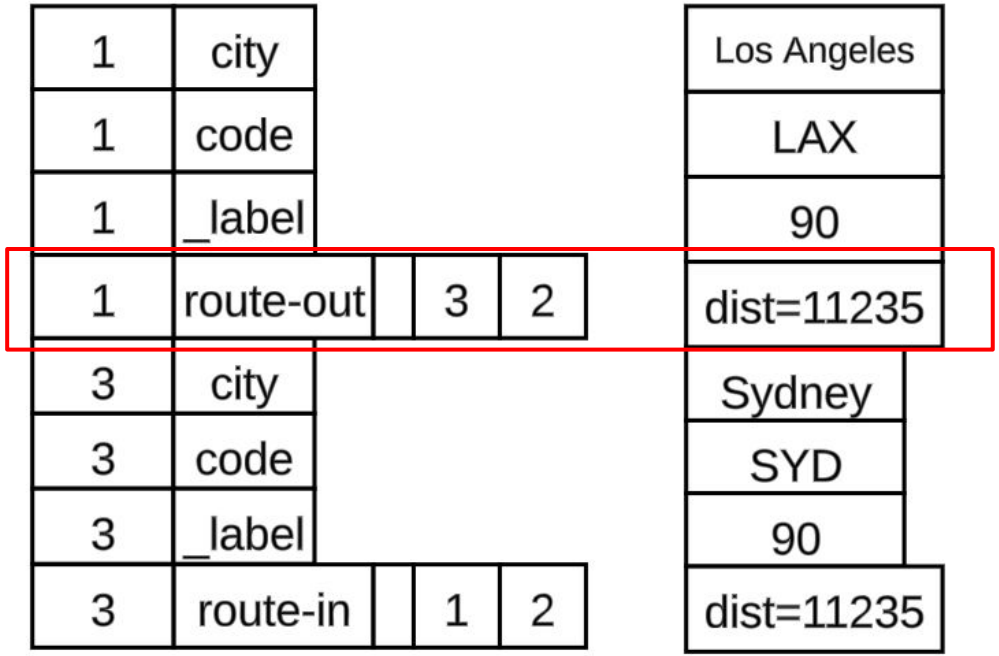
- 1 getRange



Graph Operation: Get Outgoing Edges

g.V(1).outE("route").valueMap()

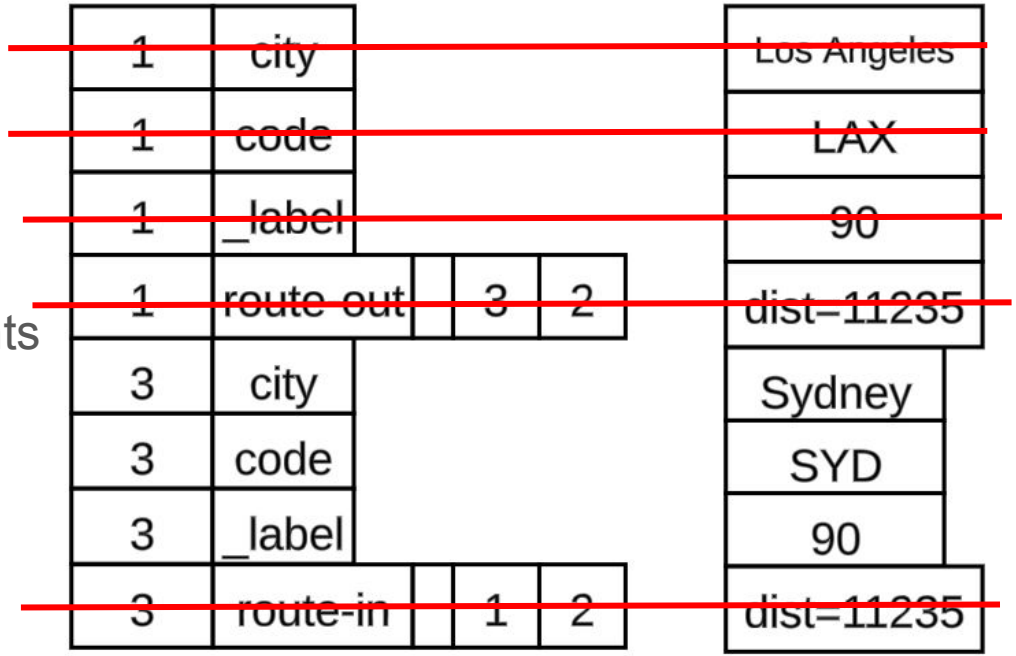
- 1 getRange (/1/route_out)



Delete a Vertex

g.V(1).drop()

- Get all edges of V1
  - Extract adjacent vertices (3)
- Remove all keys with prefix 1
  - clearRange(Range.startsWith(1))
- Remove key from 3 that represents the edge connected with 1
  - clear("3-route-in-1-2")



JanusGraph Indices

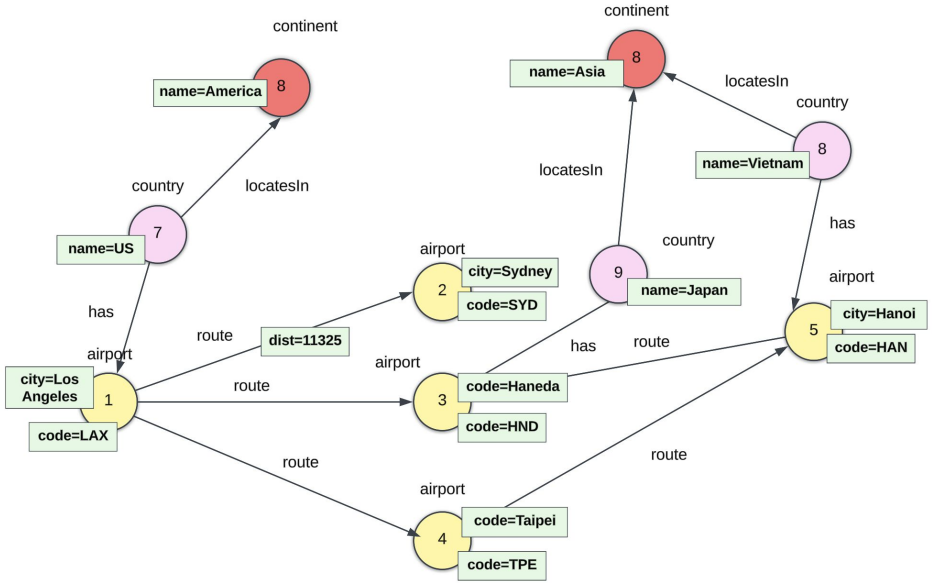
- Graph indices
  - Composite index: g.V().has("code", "LAX")
    - Hash-based
  - Mixed indices: g.V().has("name", Text.textContains("agent"));
    - External indices
    - To support global range queries and text search
    - ElasticSearch, Solr
    - **Can we incorporate them in FDB?**
- Vertex-centric Index: g.V().has("code", "LAX").outE("route").has("distance", gt(3000)).inV()
  - Local index (per vertex)
  - Deal with supernode issue where a vertex may have millions of edges connected to it.

Composite Index Example

g.V().hasLabel("airport").has("code", "LAX")

- Composite index on label="airport" and property "code"
- One lookup (get) to find the vertex id(s)

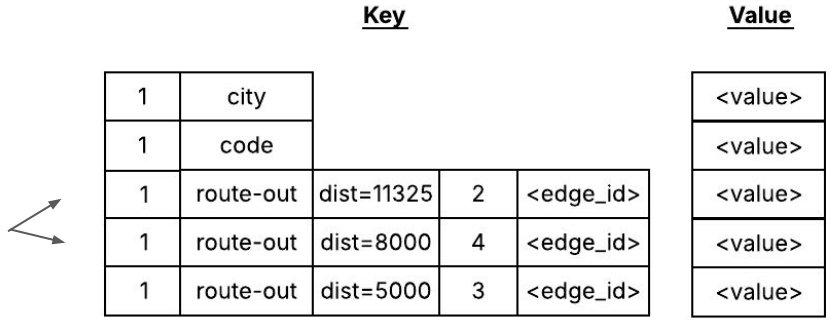
CmpIdx_airroute_LAX	1

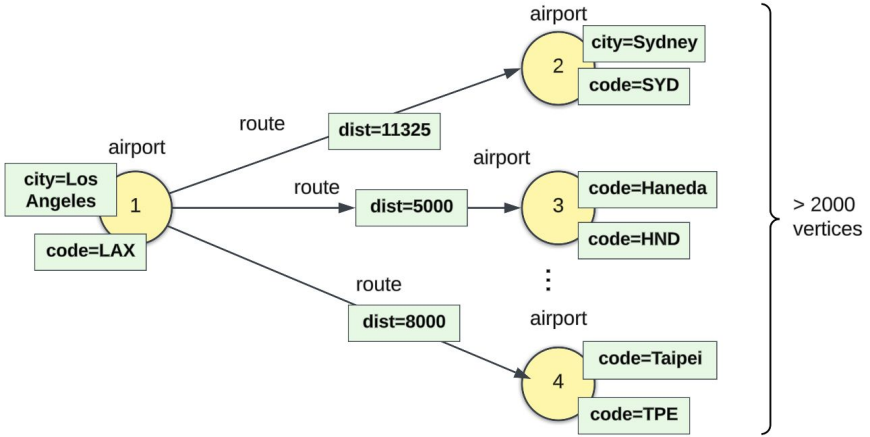


Vertex-Centric Index Example

g.V().hasLabel("airport").has("code", "LAX").outE("route").has("dist", P.lte(6000)).inV().values('code')

- Vertex-centric on edge label "route" and property "dist"
  - Edges are sorted by "dist" in FDB
- getRange for outE with a narrower range
- Deal with traversing supernode

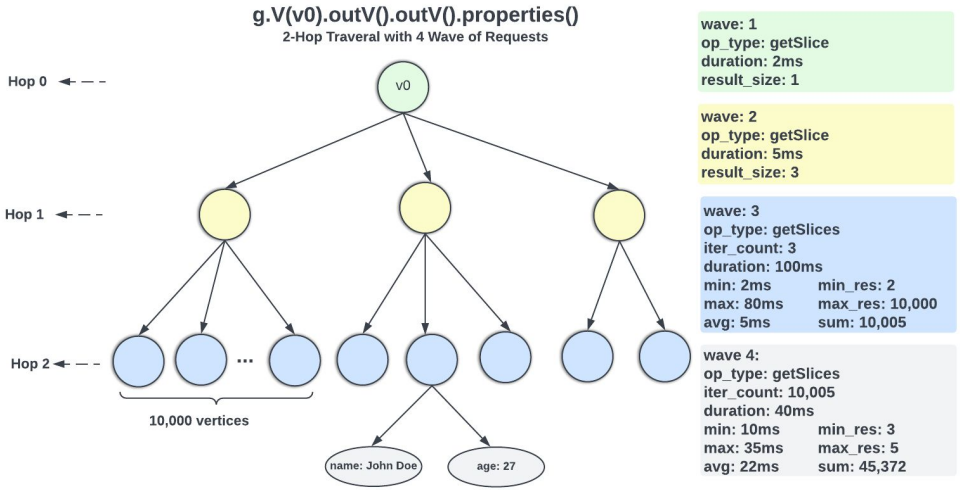




Part 3. Caching Techniques in Graph Database

Multi-hop Queries

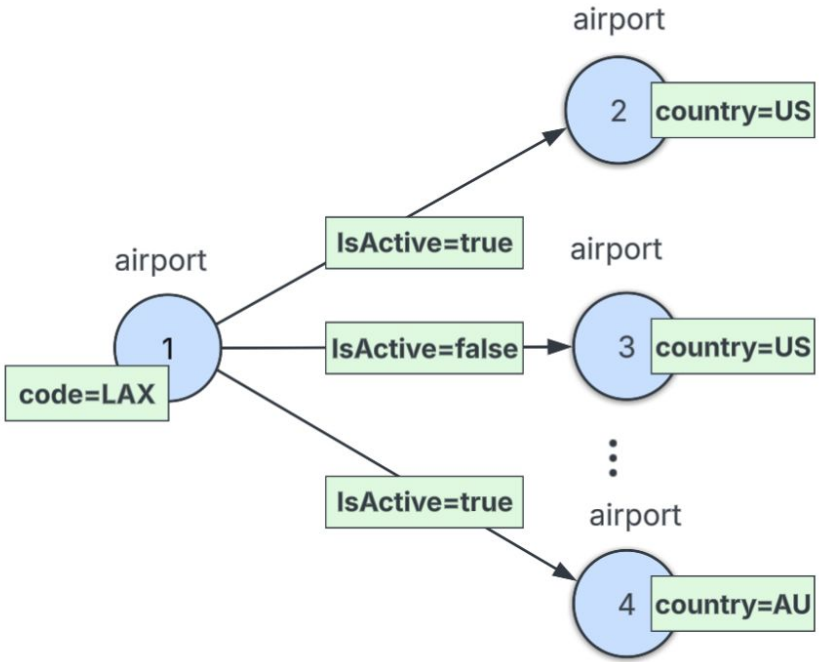
- A graph query can have multiple waves of requests
- In each wave, multiple getRange() requests are sent in parallel (in one or multiple batches) to the backend data store.
- The latency in a wave depends on the number of getRange() requests and the latency of each request.
- One large request wave typically dominates graph query latency.
- Query profiling shows the total number of waves, and statistics for each request wave



High Latency Scenario 1

- LAX is a supernode
- A large number of its neighbors are examined
- Graph processing:
  - 1 **getRange** to look up the index for vertex id with code='LAX'
  - 1 **getRange** (big) to fetch all edges of LAX
  - Deserialize byte data to get the edges and filter the ones with IsActive=true
  - Collect the neighbor vertex ids
- Cache approach:
  - Query result (neighbor vertex ids) is stored in a cache entry
    - Key: <QueryTemplate>:1:IsActive=true
  - Next time the same query comes:
    - 1 **getRange** to look up the index for vertex id with code='LAX'
    - Look up the cache to collect the neighbor vertex ids

Find all airports with active flight routes from LAX
g.V().has('code', 'LAX').outE().has('IsActive', 'true').inV()



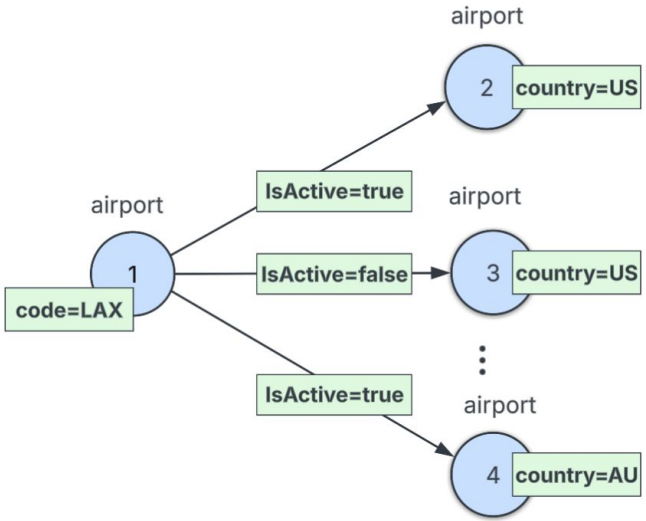
High Latency Scenario 2

- For the filtered neighbors from Scenario 1, filter their properties for country='AU'
- Graph processing:
  - Issue one getRange request for each neighboring vertex (based on its id) to fetch the property and filter the one with country='AU'
  - 1 + 1 + (N extra getRange requests sent in parallel)

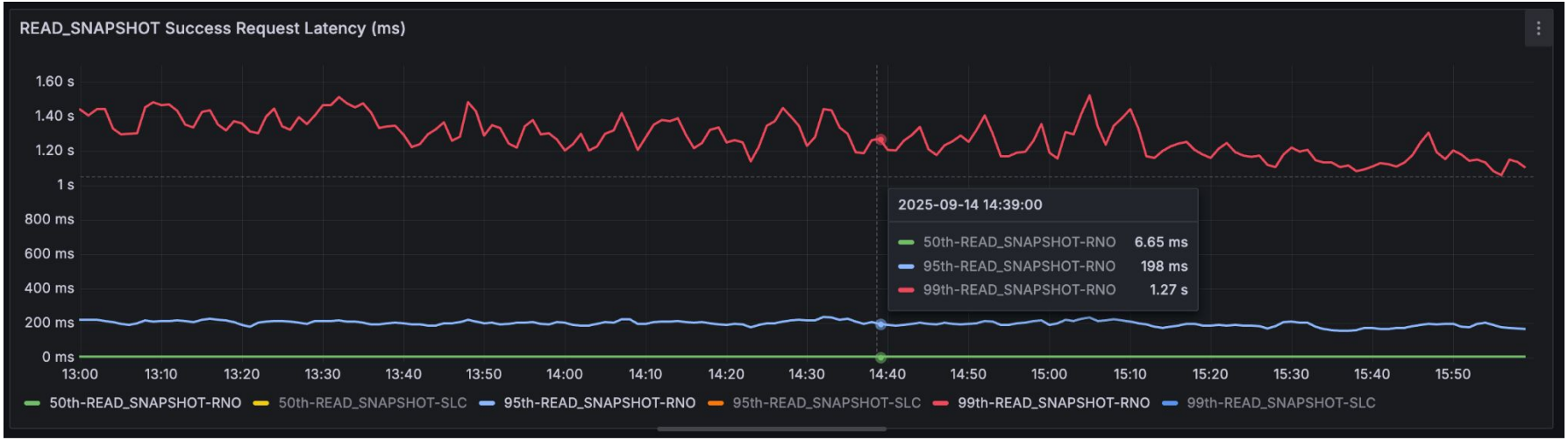
Cache approach:

  - Query result (AU airports) is stored in one cache entry
    - Key: <QT>:1:IsActive=true&Country=AU
  - 1 getRange + 1 cache lookup

Find all AU airports with active flight routes from LAX
g.V().has('code', 'LAX').outE().has('IsActive', 'true').inV().has('country', 'AU')



Without a Cache: High p95 and p99 Latency



Per-Hop Subquery Result Cache

- Divide the query into subqueries.
- Select the subqueries that dominate the original query latency.
- Process each query segment using either graph traversal or the cache.
- Example:

Find all countries with direct flights to AU airports that LAX also flies to directly.

```
g.V().has('code', 'LAX')
      .outE('route').has('IsActive', 'true').inV().has('country', 'AU')   // first-hop cache
      .inE('route').outV().values('country').dedup()                      // second-hop cache
```

- Cache Template 1: outE('route').has('IsActive', ?).inV().has('country', ?)
- Cache Template 2: inE('route').outV().values('country')
- ***Why dedup() is not in Cache Template 2?***

H. Nguyen, J. Li, and S. Ghandeharizadeh, *One-hop subquery result caches for Graph Database Systems*. Link

```
g.V().has('code', 'LAX')
      .outE('route').has('IsActive', 'true').inV().has('country', 'AU')
      .inE('route').outV().values('country').dedup()
```

Without cache:

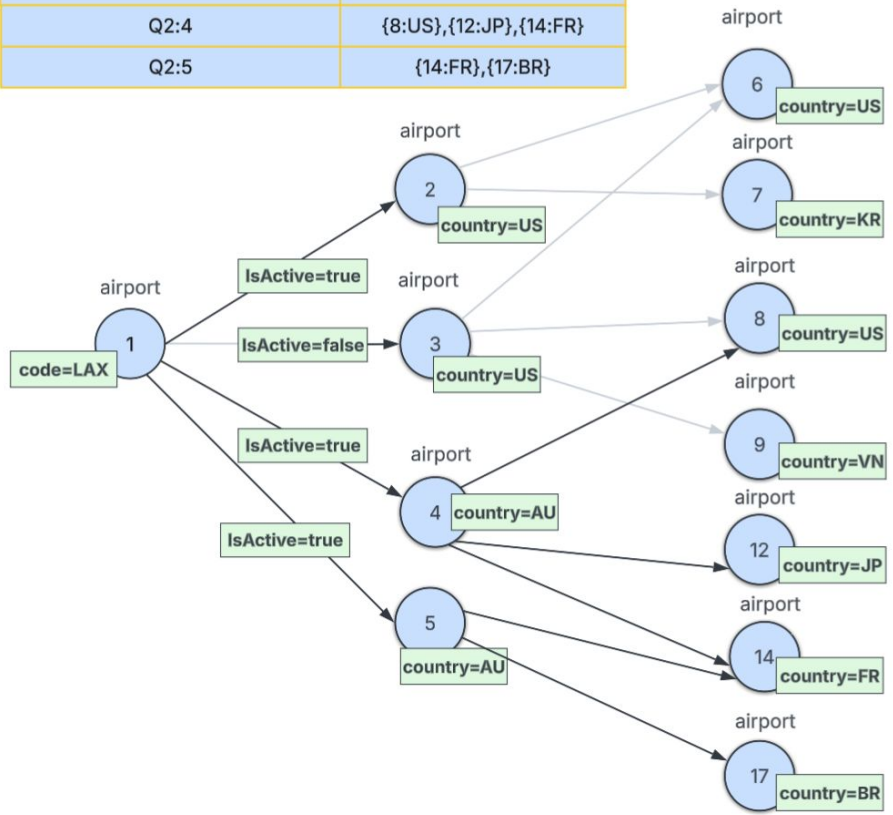
- 1 getRange for index lookup
- 1 getRange for fetching LAX edges
- 4 getRange for fetching 'country'
- 2 getRange for fetching edges
- 4 getRange for fetching 'country'

With cache:

- 1 getRange for index lookup
- 1 cache lookup:
  Q1:1:IsActive=true&country=AU
- 2 cache lookups for Q2:4 and Q2:5

Cache Entries

Q1:1:IsActive=true&country=AU	4,5
Q2:4	{8:US},{12:JP},{14:FR}
Q2:5	{14:FR},{17:BR}



One Large Database Range Read vs. One Cache Lookup

Mode	Average (ms)	95th (ms)	99th (ms)	Latency Reduction
getRange	9.7	11.4	12.5	
FDB cache get	2.2	2.2	2.2	~80%
memcached cache get	1.2	1.24	1.25	~87%

Table 1. Benchmarking 1 big getRange() vs 1 cache get() (10k values in size)

Parallel Database Range Reads versus Cache Lookups

Mode	Average (ms)	95th (ms)	99th (ms)	Latency Reduction
getRange	221	231	253	
FDB cache get	141	143	158	~36%
memcached cache get	54	54	54	~76%

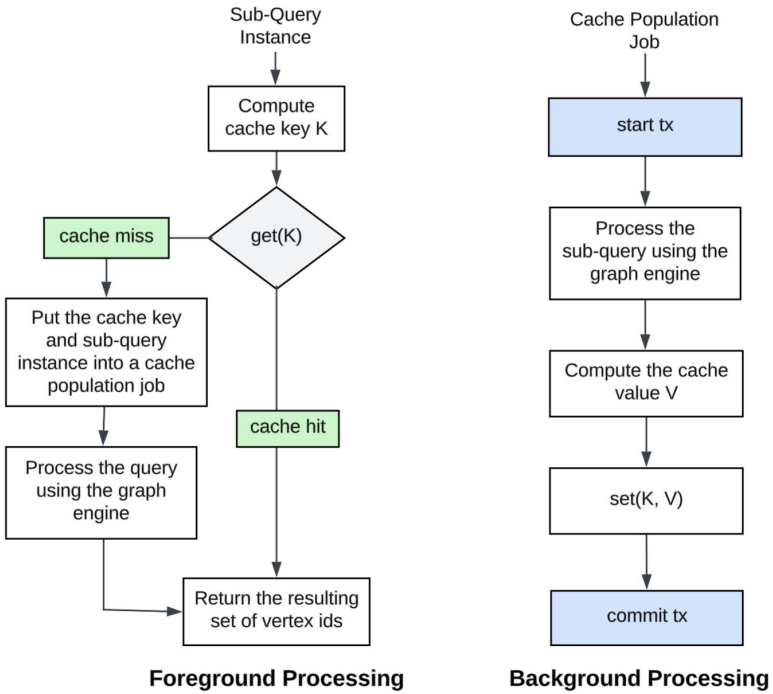
Table 2. Benchmarking 10k small getRange() vs 10k cache get()

Read Path with Cache

Given a graph traversal query:

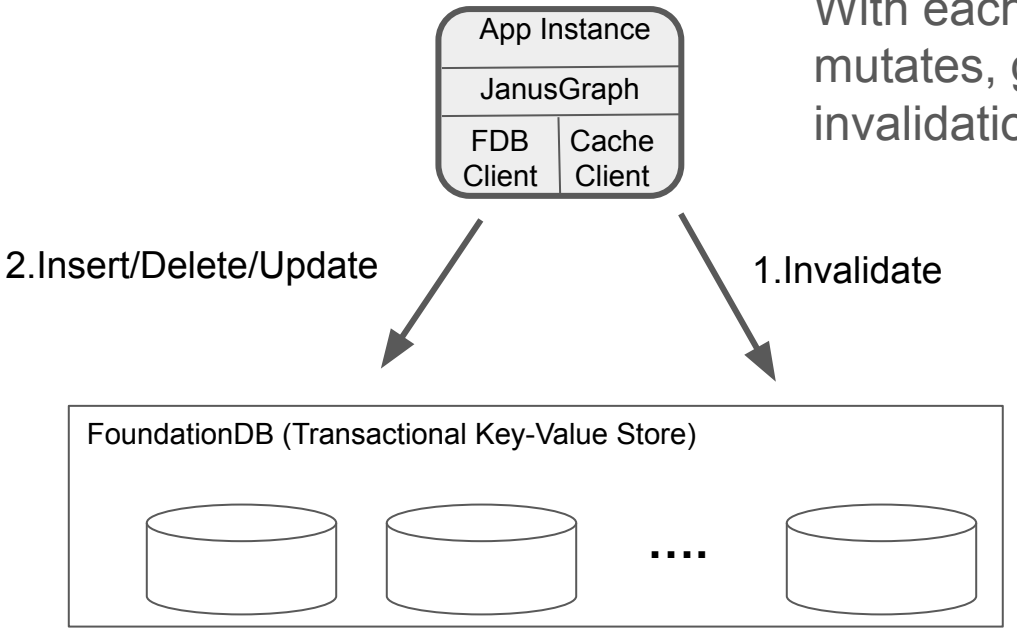
1. Decide to cache/not cache based on a predefined set of query templates
2. Compute the corresponding cache key. Look up the cache. If the value is there, return the value as the query result
3. If the value is missing, process the query via JanusGraph and push the cache key and sub-query instance to a queue.
4. Return the query result

Background cache population job populates the cache entries: query the sub-query instance and set cache value in one transaction.



S. Ghandeharizadeh, J. Yap, and H. Nguyen. 2014. Strong Consistency in Cache Augmented SQL Systems. In Proceedings of the 15th International ACM Middleware Conference, ACM Middleware 2014.

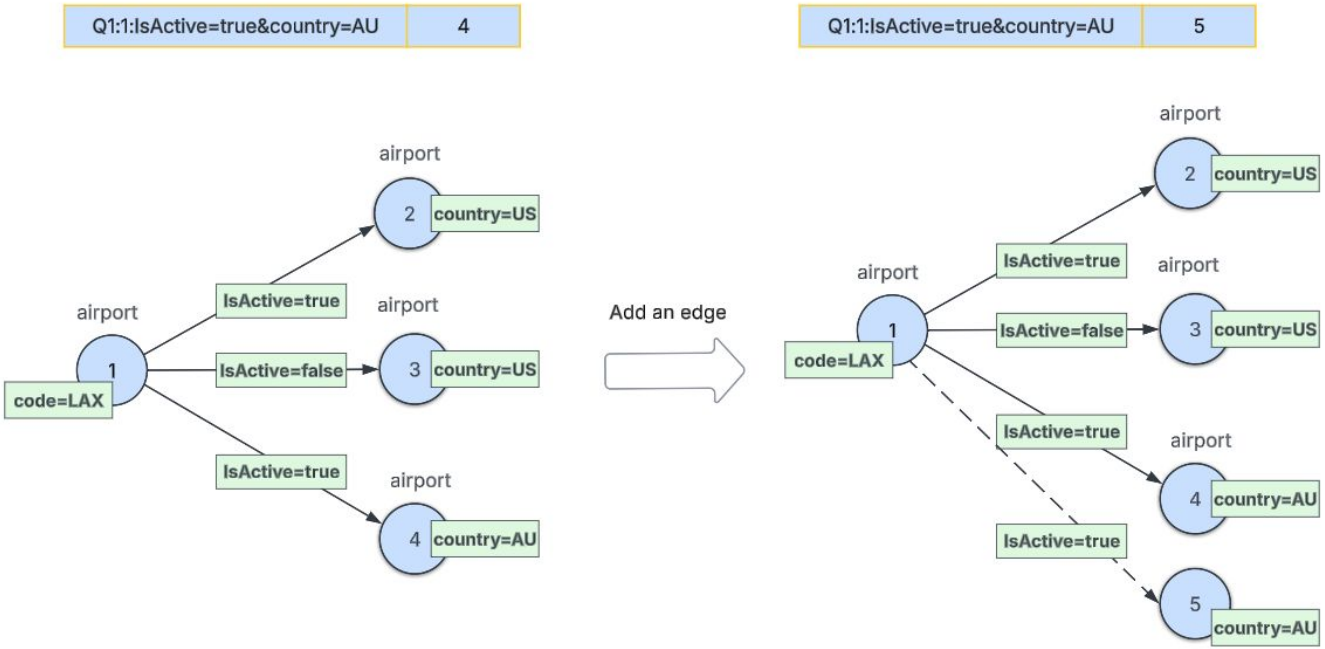
Write Path with Cache (Write-Around)



With each graph traversal that contains mutates, graph mutations and cache invalidations are in the same transaction

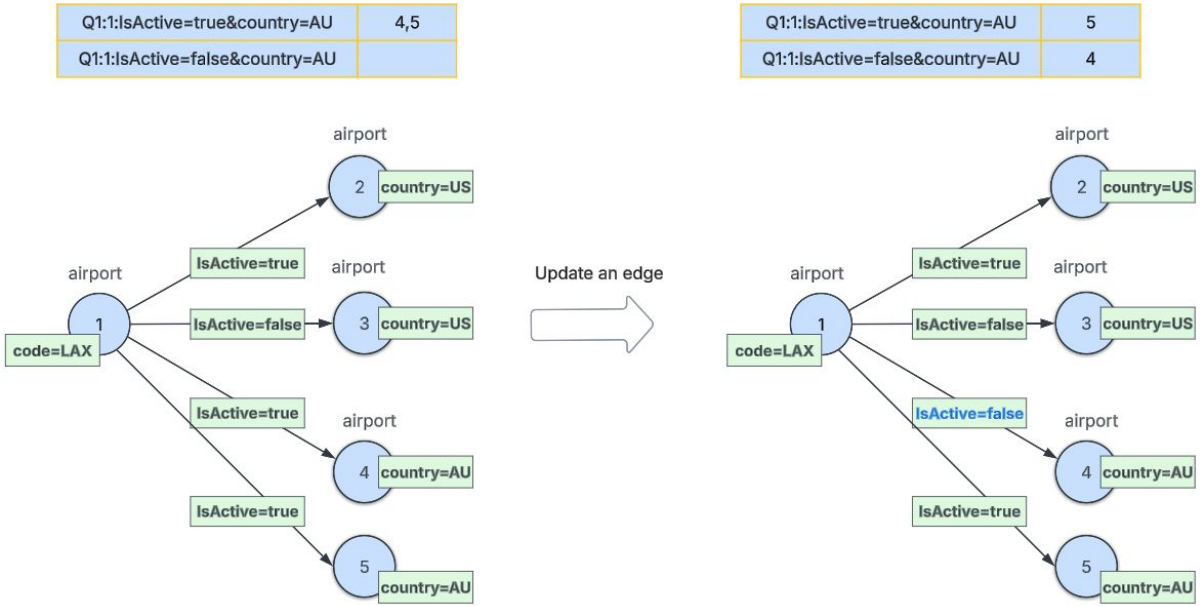
S. Ghandeharizadeh, J. Yap, and H. Nguyen. 2014. Strong Consistency in Cache Augmented SQL Systems. In Proceedings of the 15th International ACM Middleware Conference, ACM Middleware 2014.

Case 1: Add/Remove an Edge



- Impact at most 2 cache keys (per cache template)

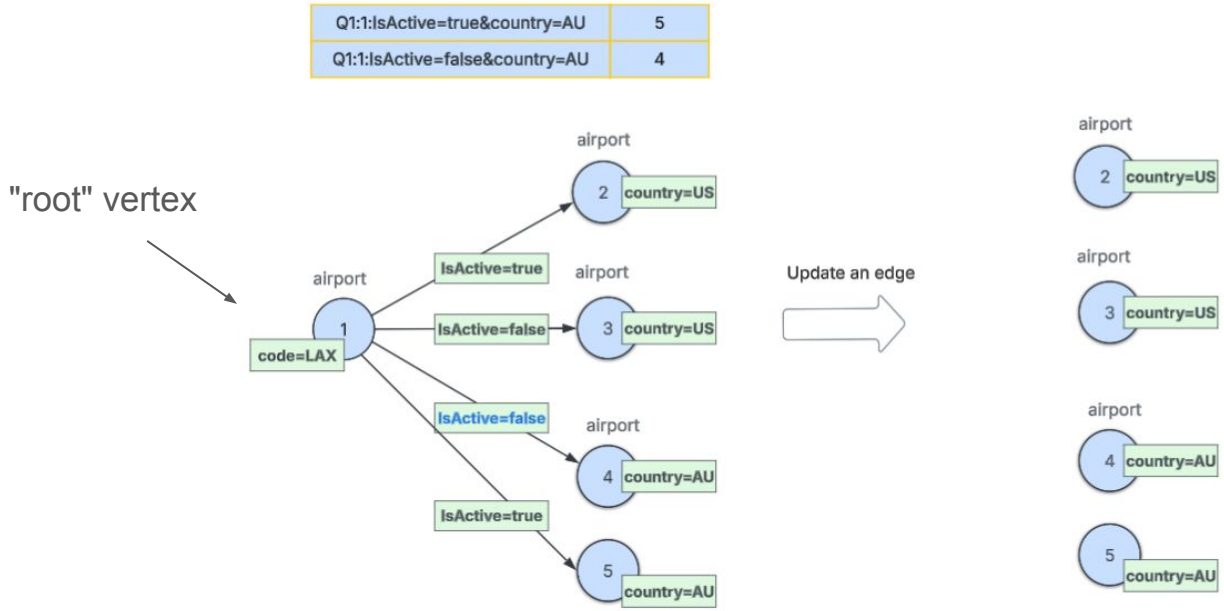
Case 2: Update an Edge



- Impact at most 4 cache keys (per cache template)

Case 3: Update/Remove a Root Vertex

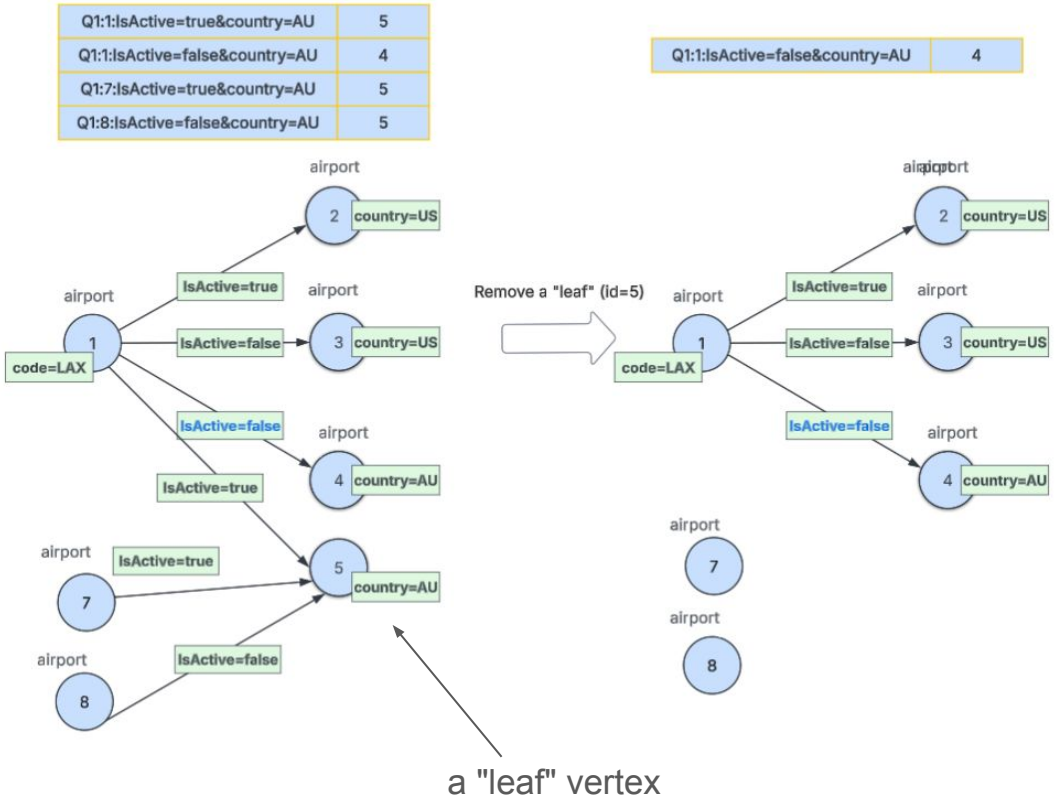
clearRange(Q1:1)



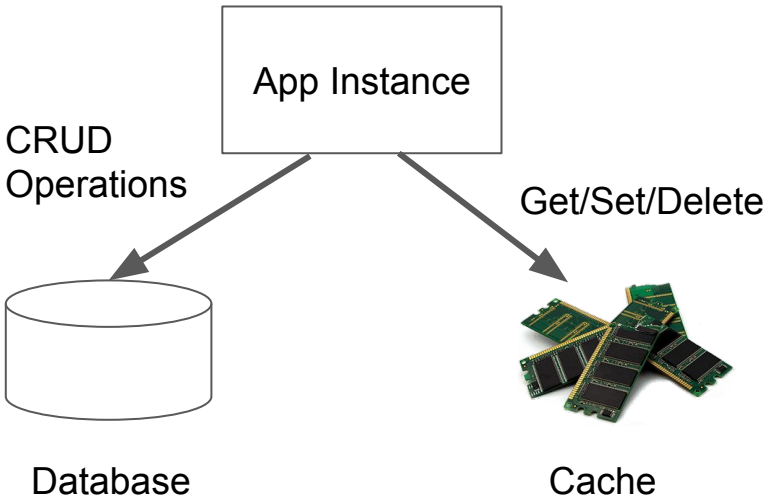
- Impact at most N cache keys (per cache template)
- Quick to invalidate based on prefix

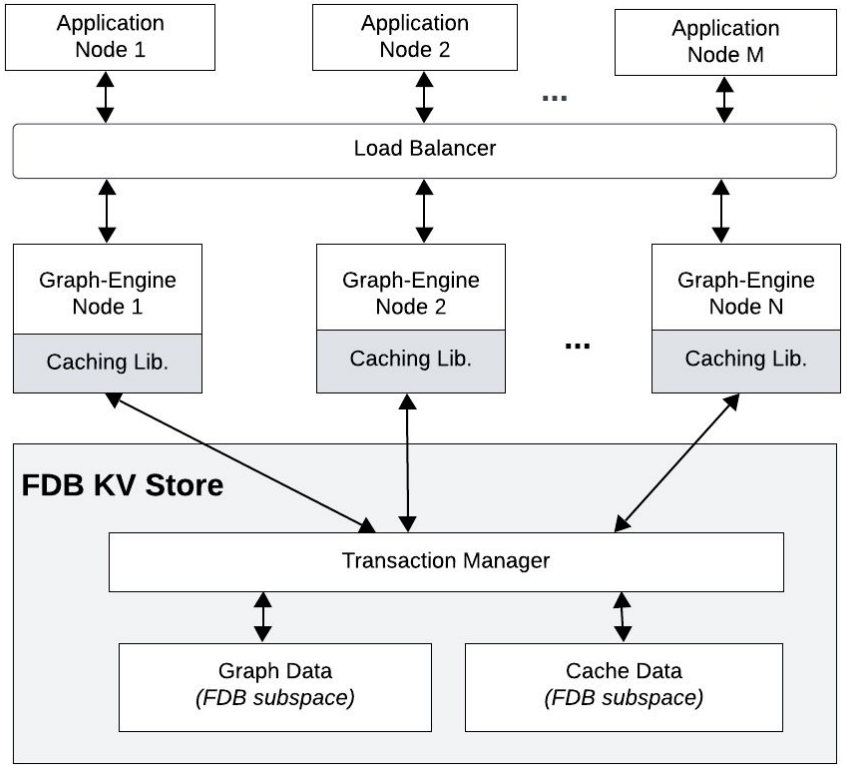
Case 4: Update/Remove a Leaf Vertex

- More expensive (up to 2N impacted cache keys)
- Exists today when an application deletes a supernode vertex
- Additional constraints limit the cost of this case:
  - If a vertex needs to be removed, its edges must be removed in small batches first
  - If a supernode vertex property needs to be updated?



FoundationDB as Cache Storage





Per-Hop Subquery Result Cache: Summary

- Significant reduction in read-only query latency
- Reasonable overhead on write queries
- Each transaction includes both operations for graph data and cache data, ensuring strong consistency in read-write workloads

Full-Query Result Cache

Query Template: g.V().has('name', ?).has('age', P.between(3, 10))
.outE('knows').has('year', P.gt(?))
.inV().where(outE('locates_at').inV().has('location', ?))
.limit(100).valueMap(true)

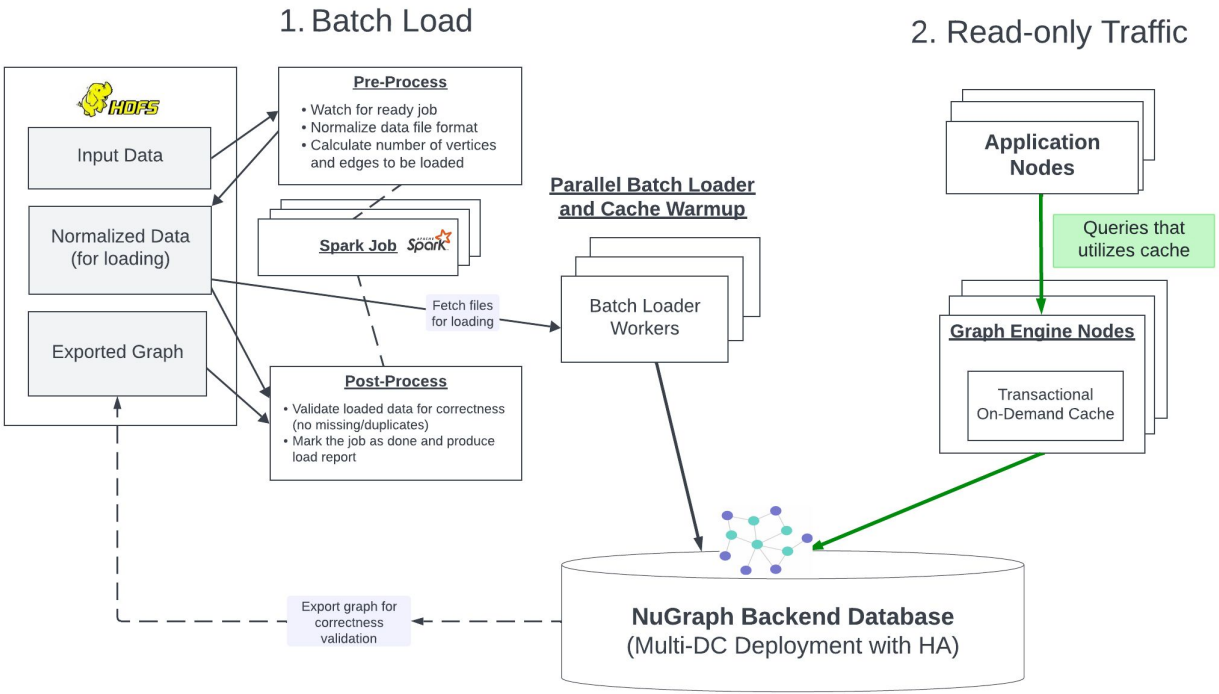
Query: g.V().has('name', 'john doe').has('age', P.between(3, 10))
.outE('knows').has('year', P.gt(20))
.inV().where(outE('locates_at').inV().has('location', 'US'))
.limit(100).valueMap(true)

Cache Key: FQ1:<john doe>:20:<US>

- Pros: Straightforward read path with substantial latency reduction
- Cons: Cache invalidation is very expensive
- Used at eBay for workloads that batch-load data before serving read-only traffic

H. Nguyen, J. Li, and S. Ghandeharizadeh. *Graph Stores with Application-Level Query Result Caches*, TPCTC 2023. Link

Batch-Load and Read-only Traffic Pipeline



Machine Learning applications

Summary

- FoundationDB is a distributed key-value store that is horizontally scalable and strongly consistent
- FDB uses MVCC and OCC to manage transactions
- Its design follows a "divide and conquer" approach where each module is responsible for a specific task
  - Each module can scale independently
  - Each module can have special hardware tailored for the module's needs
  - Easier to detect bugs and debug
- eBay has developed a graph database service using FDB as the storage backend and successfully managed graphs with billions of vertices and edges
- Cache solutions are practical for graph and significantly reduce query latency

References

1. FoundationDB paper: https://www.foundationdb.org/files/fdb-paper.pdf
2. JanusGraph doc: https://docs.janusgraph.org/
3. TinkerPop doc: https://TinkerPop.apache.org/docs/current/reference/
4. Gremlin query language: https://arxiv.org/abs/1508.03843
5. Practical Gremlin book: https://www.kelvinlawrence.net/book/PracticalGremlin.html

Q & A



Consistency Models

- Strict serializability
- Serializability
- Linearizability
- Read snapshot
- Eventual consistency
- Causal consistency
- Read-your-write consistency

Graph Databases

- Amazon Neptune
- DGraph
- **JanusGraph**
- NebulaGraph
- Neo4J
- TigerGraph

JanusGraph Quiz

What is the storage overhead of adding one vertex-centric index for an edge property?

Does composite index support range query?

Does vertex-centric index support range query?

Does mixed index support range query?